Neural ODEs

Scientific Machine Learning and Simulation Intelligence

Francesca Cairoli

University of Trieste, 2026/27

Introduction

Neural ODE is a type of neural network that combines concepts from ordinary differential equations (ODEs) and deep learning.

Neural ODEs can be viewed in two ways:

1. A neural network trained using a numerical ODE solver
2. A black-box system modeling technique that happened to use a neural net to approximate the system's differential equation (i.e. its governing equations).

Ordinary Differential Equation (ODE)

An ODE describes how the state of a dynamical system changes with respect to a continuous independent variable, typically time.

The change in time is described via a derivative $\frac{d\mathbf{x}(t)}{dt} = f(t, \mathbf{x}(t); \theta)$, — **parametric ODE**.

Given an initial condition $\mathbf{x}(t_0) = \mathbf{x}_0$ (at which point the process starts) we can solve the ODE and see how the process will evolve up to some final state.

The **solution** is also called the integral curve (because we can integrate the equation to get the solution $\phi(t)$).

Existence and Uniqueness of Solutions (Sufficient NOT Necessary Conditions!)
A unique solution exists over any finite time interval I if:

- $f_\theta(t, \mathbf{x})$ is continuous with respect to time t.
- $f_\theta(t, \mathbf{x})$ is globally Lipschitz continuous with respect to the state $\mathbf{x}$, uniformly in t, i.e., there exists $L > 0$ such that

$$\|f_\theta(t, x_1) - f_\theta(t, x_2)\| \leq L\|x_1 - x_2\|, \quad \forall t \in I, \forall x_1, x_2 \in \mathbb{R}^n .$$

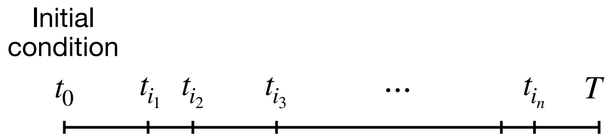
Remark: the existence and uniqueness of an exact solution do not, by themselves, guarantee that a numerical solver converges to it!

ODE solution

```
dsol = solve_ivp(
    fun=f,                      # ODE right-hand side: dx/dt = f(t, x)
    t_span=(t0, T),             # Integration interval
    y0=x0,                     # Initial condition
    method="RK45",             # Numerical integration method
    t_eval=times,               # Times at which to return the solution
    args=(theta,),              # Additional parameters passed to f
    rtol=1e-3,                  # Relative error tolerance
    atol=1e-6                   # Absolute error tolerance
)
```

A typical **ODE solver requires**:

- initial condition $\mathbf{x}_0 = \mathbf{x}(t_0)$
- a sequence of time points $t_{i_1}, \ldots, t_{i_n}$ for which to solve for y
- function f that represents $d\mathbf{x}/dt$ (the differential equation)

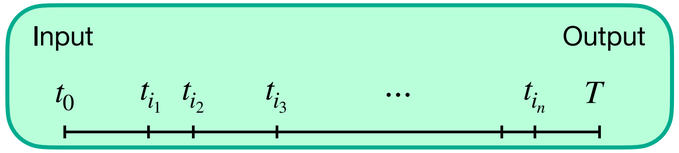


In a **Neural ODE:**

- the neural network will be the function f,
- the input will be the initial condition,
- the sequence of time points will represent how many evaluation points the neural ODE has.

$$\frac{d\mathbf{x}}{dt} = f(\mathbf{x}, \theta, t) = NN_\theta(t, \mathbf{x})$$

$$\mathbf{x}_0 = \mathbf{x}(t_0)$$



Euler Method

ODEs can be solved analytically if given in the appropriate form, but normally they are solved numerically. One of the simplest algorithms is the **Euler method**: the core idea is approximating the solution function step by step using tangents lines.

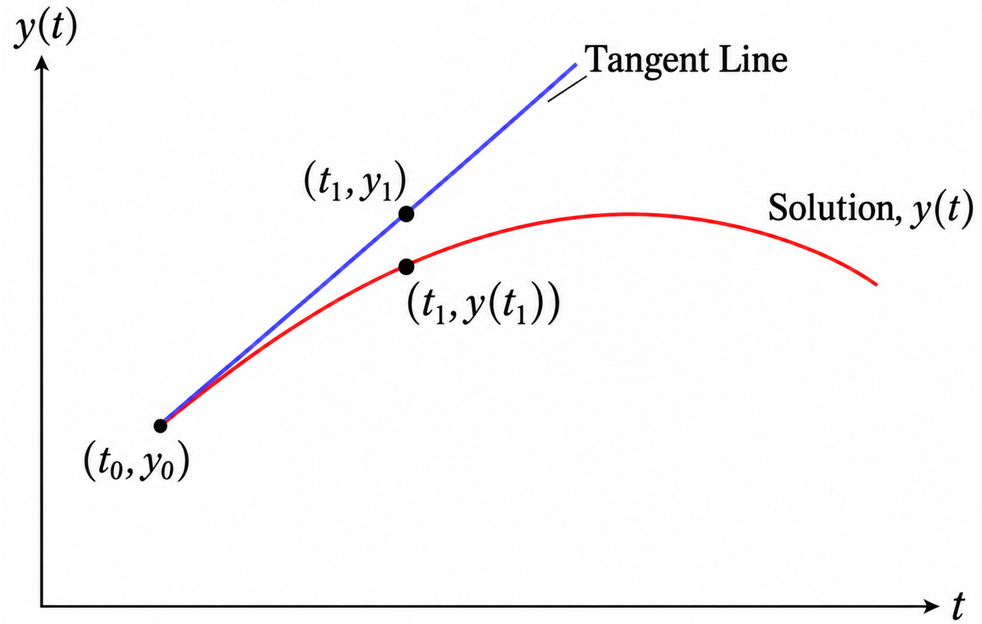
For equation

$$\frac{d\mathbf{x}}{dt} = f(t, \mathbf{x}) \qquad \mathbf{x}(t_0) = \mathbf{x}_0$$

the solution at the discretized grid of n time steps is

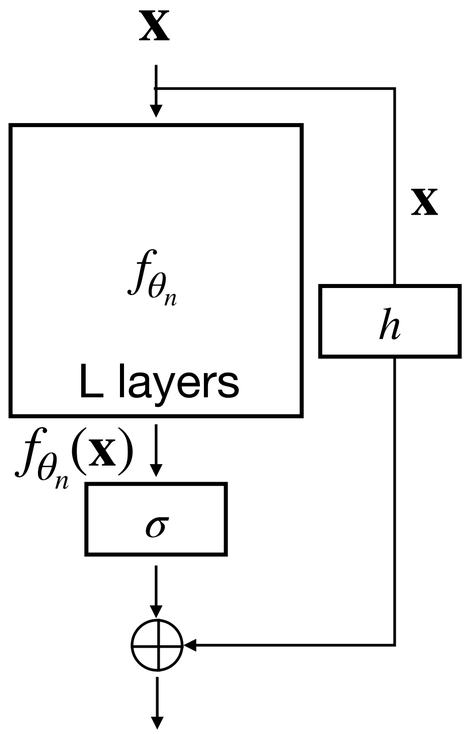
$$\mathbf{x}_{n+1} = \mathbf{x}_n + f(t_n, \mathbf{x}_n) \cdot (t_{n+1} - t_n)$$

$NN_\theta(t, \mathbf{x})$



ResNet

Assumption that input and output have the same dimension!



Residual Neural Network (ResNet) is a deep learning model that consists of **layers grouped in residual blocks**. Each block is formed by a set of layers and a shortcut connection.

If there are N residual blocks with L layers each, the n-th block computes

$$\mathbf{z}_n = h(\mathbf{x}_n) + f_{\theta_n}\mathbf{x}_n)$$
$$\mathbf{x}_{n+1} = \sigma_n(\mathbf{z}_n)$$

where $\mathbf{x}_n$ and $\mathbf{x}_{n+1}$ are the input and output of the block, respectively.

- $f_{\theta_n}(\mathbf{x}_n)$ is the output of a set of layers with parameters $\theta_n = \{\theta_{n,l} | 1 \leq l \leq L\}$.
- h is a function applied to the input of the block and gets added to the residual function.
- σ_n is the activation function.

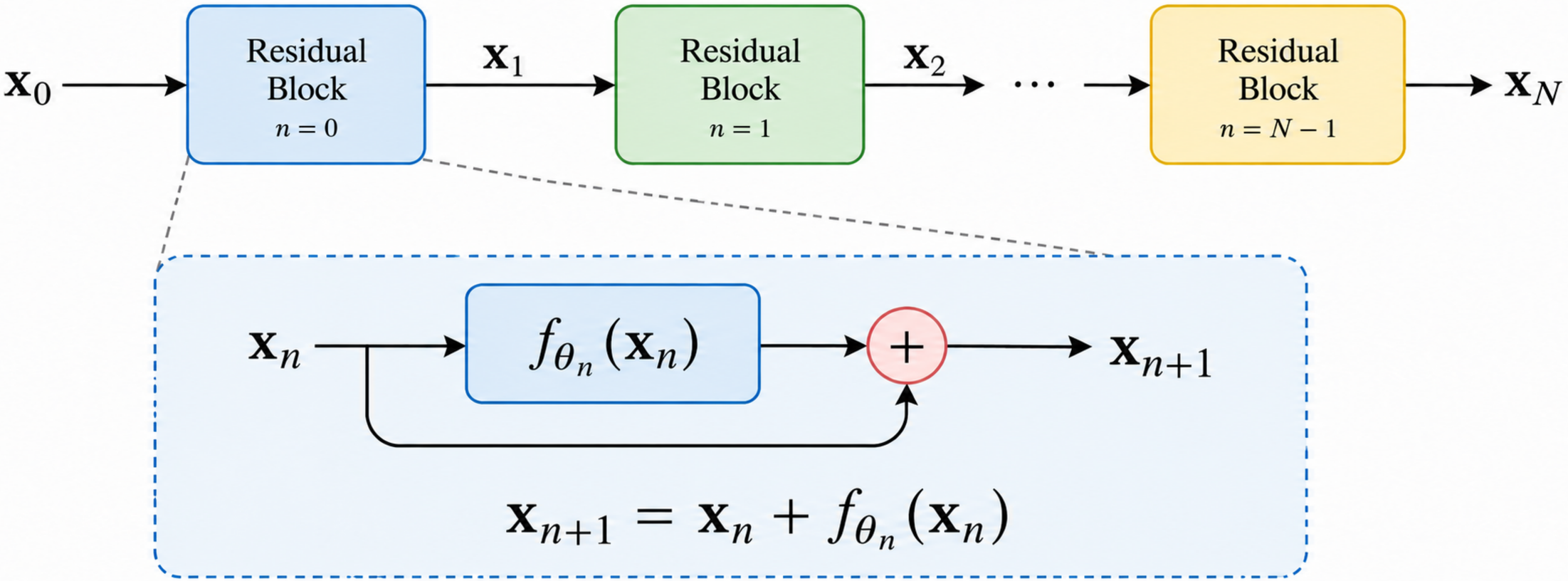
In the original paper, h was set to the identity function, and σ to the ReLu.
They later found that it is best to use the identity function for both σ and h.

$$\mathbf{x}_{n+1} = \mathbf{x}_n + f_{\theta_n}(\mathbf{x}_n)$$

Why use a ResNet?

- **Easier training of deep networks:** Skip connections improve gradient flow, mitigating vanishing gradients.
- **Feature preservation:** Identity connections allow information to pass through layers without being lost.
- **Incremental refinement:** Each block learns a correction to its input rather than an entirely new representation.
- **Improved performance:** Greater depth can improve representation capacity without making optimization prohibitively difficult.

ResNet


$\mathbf{x}_0$
Residual Block $n = 0$
$\mathbf{x}_1$
Residual Block $n = 1$
$\mathbf{x}_2$
$\cdots$
Residual Block $n = N - 1$
$\mathbf{x}_N$
$\mathbf{x}_n$
$f_{\theta_n}(\mathbf{x}_n)$
+
$\mathbf{x}_{n+1}$
$\mathbf{x}_{n+1} = \mathbf{x}_n + f_{\theta_n}(\mathbf{x}_n)$

ResNets as ODE Discretization

Residual networks can be interpreted as discrete counterparts of continuous-time dynamical systems.

Using **explicit Euler** with step size ($t_{n+1} - t_n = 1$): $\mathbf{x}_{n+1} = \mathbf{x}_n + f_\theta(t_n, \mathbf{x}_n)$.

A **ResNet block** has exactly the same structure: $\mathbf{x}_{n+1} = \mathbf{x}_n + f_{\theta_n}(\mathbf{x}_n)$.

Although standard ResNet blocks use different parameters θ_n, their transformations can be interpreted as evaluations of a time-dependent vector field at discrete time points: $f_\theta(t_n, \mathbf{x}) = f_{\theta_n}(\mathbf{x})$.

Therefore, a chain of residual blocks can be viewed as an explicit Euler discretization of an ODE, where network depth plays the role of time.

- The **initial state** $\mathbf{x}_0$ corresponds to the network **input**.
- The **final state** $\mathbf{x}_N$ corresponds to the **output** of the last residual block.

Note: This structural equivalence does not, by itself, guarantee that the ResNet accurately approximates a continuous ODE solution.

ResNets as ODE Discretization

ResNet: We first define a sequence of discrete transformations $f_{\theta_0}, \ldots, f_{\theta_{N-1}}$. A continuous-time vector field can then be constructed a posteriori to interpolate these transformations. However, this field is not uniquely determined and was not explicitly learned by the ResNet.

1. From different residual blocks to a time-dependent vector field

Consider three residual blocks with different parameters: $f_{\theta_0}(\mathbf{x}), \quad f_{\theta_1}(\mathbf{x}), \quad f_{\theta_2}(\mathbf{x})$.

We can construct a continuous time-dependent vector field by interpolating their transformations.

For example, for $t \in [0,1]$: $f_\theta(t, \mathbf{x}) = (1-t)f_{\theta_0}(\mathbf{x}) + tf_{\theta_1}(\mathbf{x})$.

Thus, $f_\theta(0,\mathbf{x}) = f_{\theta_0}(\mathbf{x})$ and. $f_\theta(1,\mathbf{x}) = f_{\theta_1}(\mathbf{x})$. The construction can be extended to the other blocks.

2. Structural equivalence does not guarantee an accurate ODE solution

Consider the ODE: $\dot{x}(t) = -x(t), \qquad x(0) = 1$, whose exact solution is $x(t) = e^{-t}$.

Euler's method gives $x_{n+1} = x_n - \Delta t x_n = (1-\Delta t)x_n$.

With $\Delta t = 1$, after one step:

$$\underbrace{x_1 = 0}_{\text{Euler / ResNet}} \qquad \text{vs.} \qquad \underbrace{x(1) = e^{-1} \approx 0.368}_{\text{Exact ODE}}.$$

Neural ODE: We directly learn a single continuous-time vector field $\dot{\mathbf{x}}(t) = f_\theta(t, \mathbf{x}(t))$, which can be evaluated at arbitrary times and numerically integrated to obtain the system's evolution.

Even though the residual update has exactly the structure of Euler's method, it may poorly approximate the continuous solution. Reducing Δt improves accuracy under standard convergence assumptions; simply adding ResNet blocks does not guarantee this.

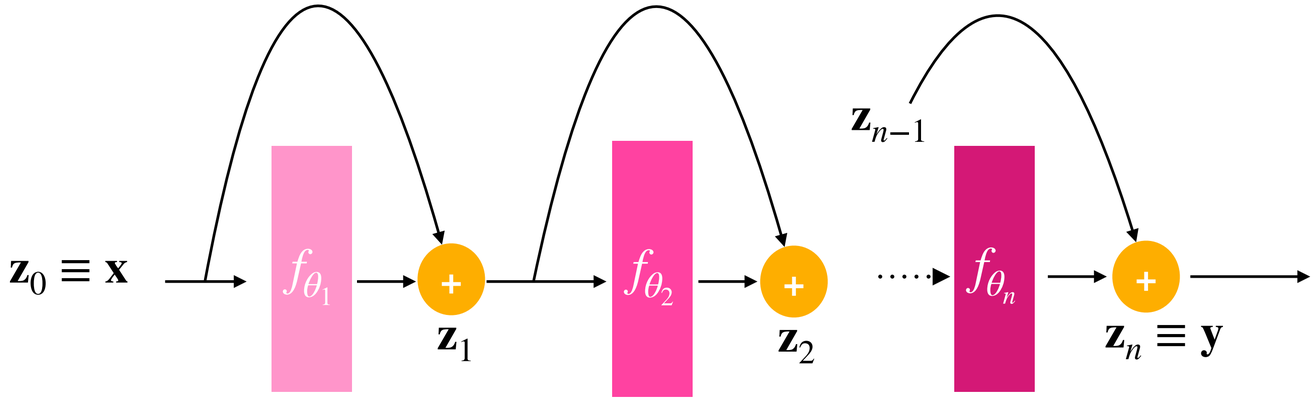
ResNets and Numerical ODEs

Standard ResNet: different parameters per block

Each residual block has its own function, weights, biases, etc.

Residual update:

$$\mathbf{z}_n = f_{\theta_n}(\mathbf{z}_{n-1}) + \mathbf{z}_{n-1}$$

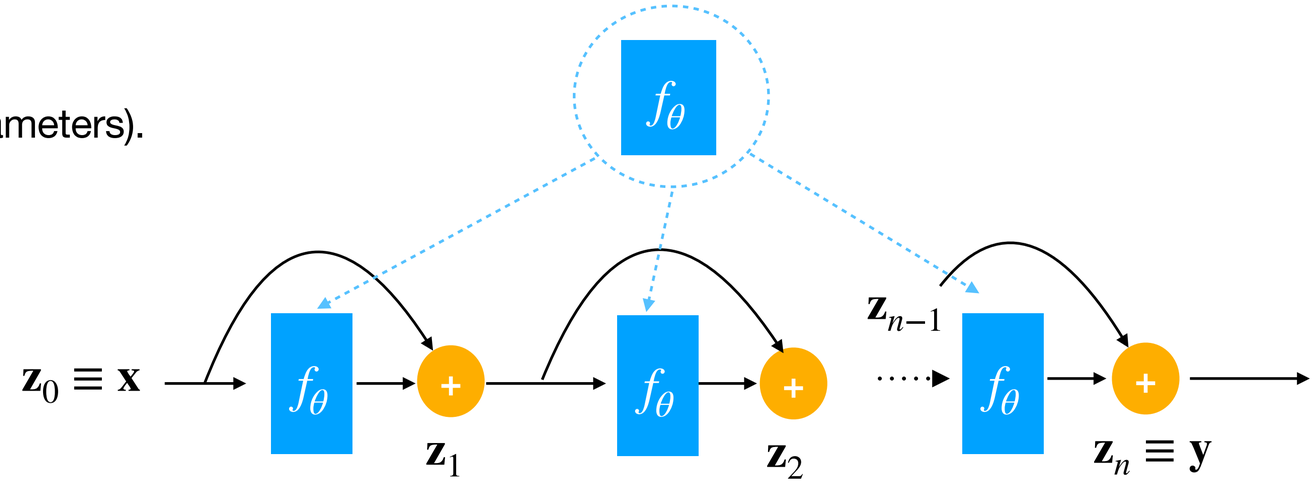


Shared-parameter ResNet: same function across layers

Consider a common function f_θ used at all layers (shared parameters).

Residual update:

$$\mathbf{z}_n = f_\theta(\mathbf{z}_{n-1}) + \mathbf{z}_{n-1}$$



Connection with Euler's method

A chain of N residual blocks has the same structure as N explicit Euler steps with step size $\Delta t = 1$:

$$\mathbf{z}_n = f_\theta(t_{n-1}, \mathbf{z}_{n-1}) + \mathbf{z}_{n-1}$$

Neural ODE

Consider the following initial value problem

$$\begin{cases} \frac{d\mathbf{x}}{dt}(t) = f_\theta(t, \mathbf{x}(t)) \\ \mathbf{x}(t_0) = \mathbf{x}_0 \end{cases}$$

where $\mathbf{x} : \mathbb{R} \to \mathbb{R}^d$, $t \in \mathbb{R}$, $f_\theta : \mathbb{R} \times \mathbb{R}^d \to \mathbb{R}^d$ and $\theta \in \mathbb{R}^p$.

The IVP is said to be a **Neural Ordinary Differential Equation** if the function f_θ is a **neural network**, i.e. the differential equation's vector field is parameterized using a neural network. Here, f_θ can be any neural architecture.

As it is, there is no guarantee that the solution is unique or that it even exists.

To ensure that a Neural ODE defines a unique trajectory, the vector field $f_\theta(t, \mathbf{x})$ is typically assumed to be continuous in time and locally Lipschitz continuous in the state $\mathbf{x}$.

Existence and Uniqueness of Neural ODE solution: Standard neural networks with finite weights and Lipschitz activations (e.g., ReLU, tanh, sigmoid) are globally Lipschitz with respect to their inputs.

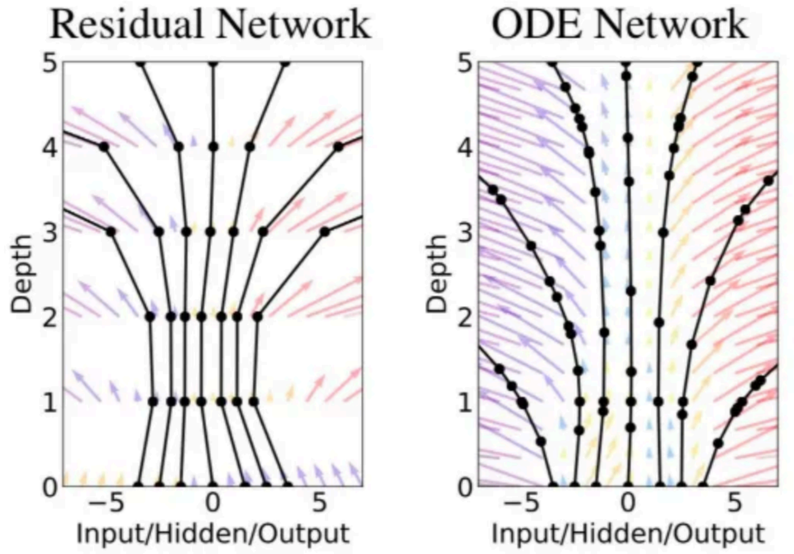
Neural ODE

$$\begin{cases} \frac{d\mathbf{x}}{dt}(t) = NN_\theta(t, \mathbf{x}(t)) \\ \mathbf{x}(t_0) = \mathbf{x}_0 \end{cases}$$

Now, instead of thinking of a network as a finite sequence of residual blocks, we can think of it as a continuous dynamical system.

We use a neural network to define the derivative of the hidden state with respect to time.

Given the current hidden state and, optionally, the current time, the neural network tells us how that state should change.



Difference between ResNet with a fixed number of layers and ODENet with a flexible number of layer

Neural ODEs as the Continuous-Depth Limit of ResNets

Neural ODEs can be interpreted as the **continuous-depth limit of appropriately scaled ResNets**, where the number of residual blocks increases ($N \to \infty$) while the step size decreases ($\Delta t \to 0$) over a fixed interval.

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \Delta t\, NN_\theta(t_n, \mathbf{x}_n) \quad \xrightarrow[\Delta t \to 0]{} \quad \frac{d\mathbf{x}(t)}{dt} = NN_\theta(t, \mathbf{x}(t))$$

This limit holds under suitable regularity assumptions.

Key idea: Network depth becomes a continuous variable, and the hidden state evolves according to a learned differential equation rather than through a fixed number of discrete layers.

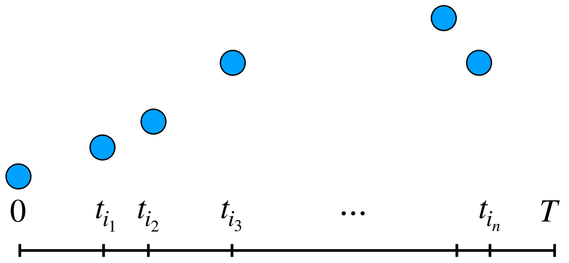
The resulting dynamics can be approximated using numerical ODE solvers, including adaptive-step methods.

Neural ODE

Neural ODEs define a family of continuous-depth neural networks, where the evolution of hidden representations is governed by a learned differential equation.

Instead of specifying a discrete sequence of hidden layers, we parameterize the derivative of the hidden state using a neural network.

The output and the resulting dynamics are integrated using a **numerical ODE solver**.



Observations available at arbitrary times

Motivations

Remark: Neural ODE time does not necessarily correspond to physical time. In a classification network, for example, it simply represents how the hidden representation evolves through the model."

- **Continuous-depth modeling:** hidden representations evolve according to a learned differential equation rather than a fixed sequence of residual blocks.
- **Adaptive computation:** numerical solvers can dynamically adjust the number of evaluations of the vector field.
- **Potential memory efficiency:** adjoint-based methods can avoid storing all intermediate forward solver states.
- **Parameter sharing:** the same vector field is evaluated throughout the integration interval, potentially reducing the number of trainable parameters.
- **Irregularly sampled data:** predictions can be evaluated at arbitrary timestamps without imposing a fixed temporal grid.

Neural ODE

Given the input layer $\mathbf{x}(0)$, the output layer $\mathbf{x}(T)$ can be defined as the solution to the IVP at some time T.

This value can be computed by a **black-box differential equation solver**, which evaluates the hidden unit dynamics f_θ wherever necessary to determine the solution with the desired accuracy.

IVP has solution $\phi : [0,T] \to \mathbb{R}^d$.

$SolveODE(NN_\theta, [0,T], \mathbf{x}_0)$ returns the solution (approximate) in a discretization of $[0,T]$.

For any $\mathbf{x} \in \mathbb{R}^d$, define the mapping $\psi : \mathbb{R}^d \to \mathbb{R}^d$ which corresponds to a flow ϕ_t

$$\mathbf{x} \mapsto \phi(T; 0, \mathbf{x}, \theta)$$

ψ is a map that, given an input vector $\mathbf{x}$, outputs a prediction corresponding to the value of the solution to the IVP with initial condition $\mathbf{x}(0) = \mathbf{x}$, at time T

Very rigid structure as that the source and target spaces are the same, so it would mean that it can only be used to predict values in the same space as the input variables.

This can be solved by composing with auxiliary functions before and after ψ.

Neural ODE

Remark: Neural ODE time does not necessarily correspond to physical time. In a classification network, for example, it simply represents how the hidden representation evolves through the model."

Example of **image classification problem**

Data MNIST dataset: input images in $\mathbb{R}^{28\times28}$, output labels in $\mathbb{R}^{10}$

Basic Model

$f_\theta : \mathbb{R} \times \mathbb{R}^{28\times28} \to \mathbb{R}^{28\times28}$ CNN

$l_\theta : \mathbb{R}^{28\times28} \to \mathbb{R}^{10}$ affine

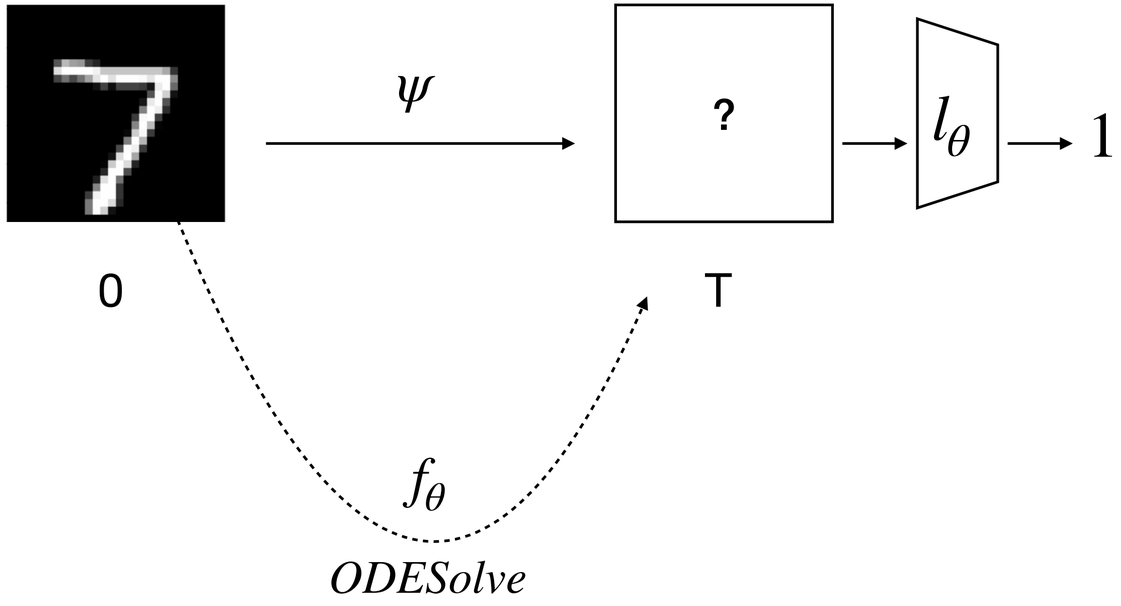
$$\Psi : \mathbb{R}^{28\times28} \to \mathbb{R}^{28\times28} \to \mathbb{R}^{10}$$

$$\mathbf{x} \mapsto \psi(\mathbf{x}) \mapsto \textbf{softmax}(l_\theta(\psi(\mathbf{x})))$$

Flow of the ODE to some time T

Optimize wrt θ to classify the input y correctly

Differentiable SolveODE



$$\mathscr{L}(\mathbf{x}, \mathbf{y}; \theta) = \text{CrossEntropyLoss}\Big(\text{softmax}(l_\theta(ODESolve(NN_\theta, \mathbf{x}, T))), \mathbf{y}\Big)$$

Differentiating through a black-box ODE Solver

Is Euler too primitive ODE solving method?

Indeed it is, so let's replace ResNet / EulerSolverNet with some abstract concept as **ODESolveNet**, where **ODESolve** will be a function, that provides a solution to the ODE. The network architecture now might look like the following:

```
nn = Network(
  Dense(...), # making some primary embedding
  ODESolve(...), # "infinite-layer neural network"
  Dense(...) # output layer
)
```

Modern ODE solvers provide guarantees about the growth of approximation error, monitor the level of error, and adapt their evaluation strategy on the fly to achieve the requested level of accuracy. This allows the ***cost of evaluating a model to scale with problem complexity***

A neural network is a differentiable function, so we can train it with gradient-based optimization routines. How should we **back-propagate through the** $ODESolve()$ function, which is a black box in our case?

We will see how to compute gradients of a scalar-valued loss with respect to all inputs of any ODE solver, ***without back-propagating through the operations of the solver***.

We will also see how not storing any intermediate quantities of the forward pass allows us to train our models with ***constant memory*** cost as a function of depth, a major bottleneck of training deep models.

Problem Statement

Let $\mathbf{z} \in \mathbb{R}^N$ be a vector whose dynamics evolve according to the ordinary differential equation

$$\frac{d\mathbf{z}}{dt} = \mathbf{f}(\mathbf{z}, \theta, t) \qquad \mathbf{f} \text{ neural net}$$

where $\theta \in \mathbb{R}^P$ and let $ODEsolve(\mathbf{z}_0, t_0, t_1, \theta)$ denote the solution to this ODE when $\mathbf{z}(t_0) = \mathbf{z}_0$.

Let $\mathscr{L} : \mathbb{R}^N \to \mathbb{R}$ be a loss function.

Goal: compute the derivatives of the loss $\mathscr{L}(ODEsolve(\mathbf{z}_0, t_0, t_1, \theta), Z_{\text{obs}})$ w.r.t. the parameters θ.

Memory-efficient training of Neural ODEs

Differentiating directly through an ODE solver may require storing a large number of intermediate activations, especially when using multi-stage or adaptive numerical methods.

Adjoint sensitivity methods offer an alternative: compute gradients by solving an additional ODE backward in time, without storing the entire forward computational graph.

Computational Graph Size

For an ODE solver performing N integration steps:

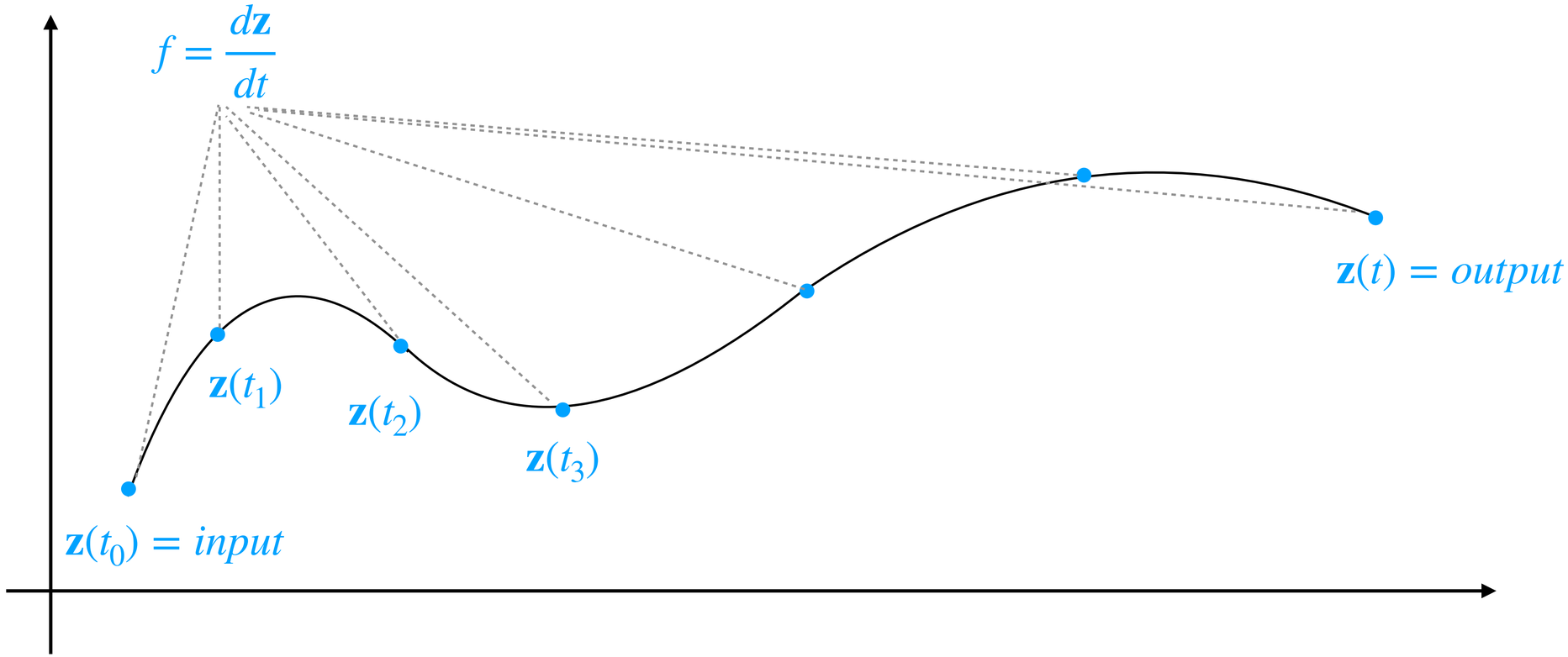
- **Euler:** N evaluations of f_θ.
- **RK4:** $4N$ evaluations of f_θ.
- **Adaptive solvers:** A variable number of evaluations, depending on the dynamics and error tolerances.

Remark

Ideally we want to compute $\frac{\partial \mathscr{L}(\mathbf{z}(t_1))}{\partial \theta}$ but the solver returns an approximation $\hat{\mathbf{z}}(t_1) \approx \mathbf{z}(t_1)$, so by differentiating through the solver we obtain $\frac{\partial \mathscr{L}(\hat{\mathbf{z}}(t_1))}{\partial \theta}$.

ODESolve

We can now see how the forward propagation of a neural ODE block look likes:



$ODESolve(\mathbf{z}_0, t_0, T, f)$ can be any solver we want.

Reverse-mode automatic differentiation of ODE solution

The main technical difficulty in training continuous-depth networks is performing reverse-mode differentiation (also known as back-propagation) through the ODE solver. Moreover, the resulting gradients correspond to the discretized solution and may differ from those of the exact continuous-time system due to numerical integration errors.

Differentiating through the operations of the **forward** pass is straightforward, but incurs a high memory cost, as intermediate activations must be stored.

NeuralODE treats the ODE solver as a black box and compute gradients using the **adjoint sensitivity method**.

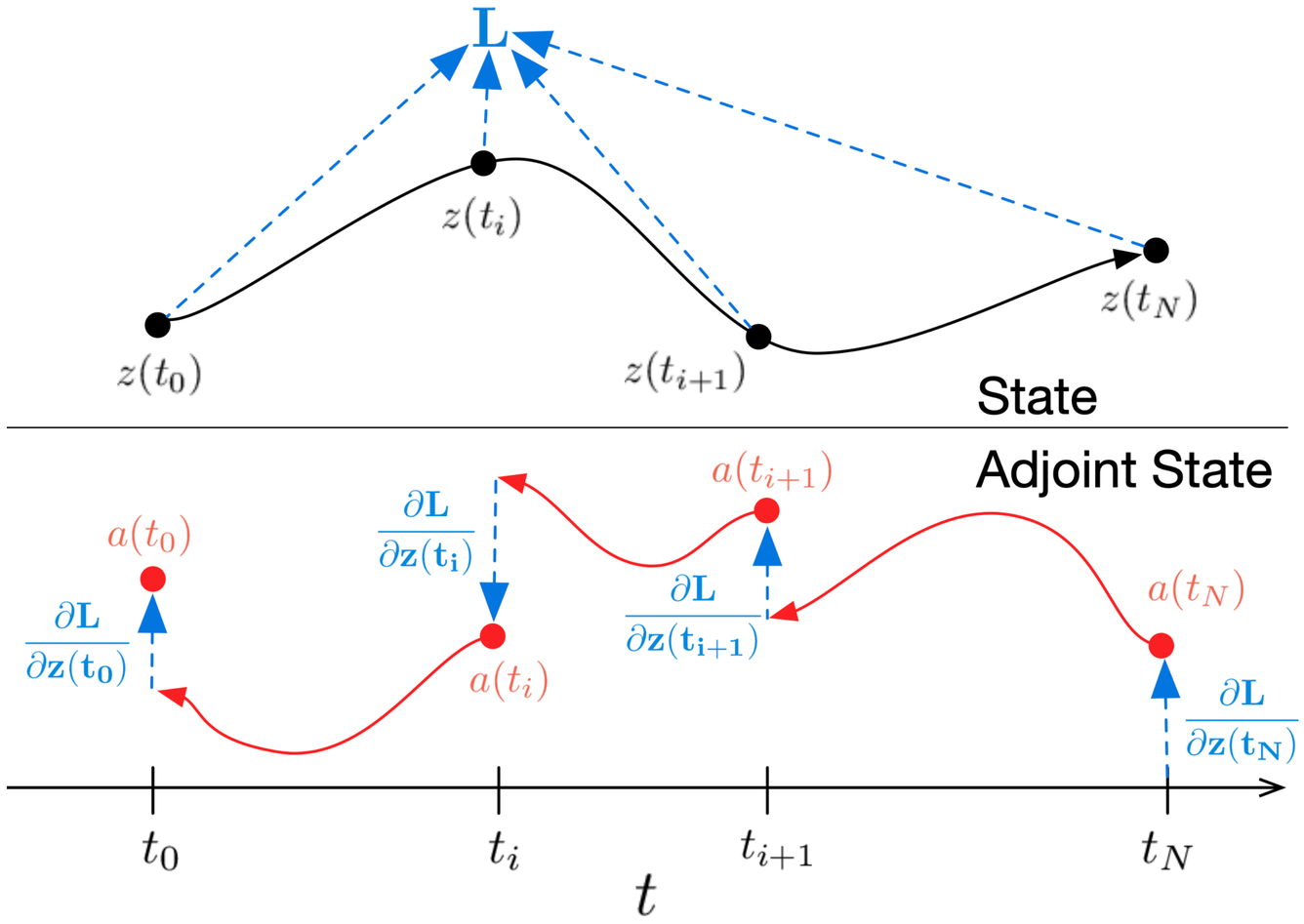
This approach computes gradients by solving a second, augmented ODE backwards in time, and is **applicable to all ODE solvers**.

This approach scales linearly with problem size, has low memory cost, and explicitly controls numerical error.

Adjoint method

Goal: compute $\frac{dL}{d\theta}$ efficiently

We need the gradient of loss function wrt the input and the dynamics parameters. The mathematical trick is called adjoint sensitivity method.



- L stands for the main loss function we want to optimize.
- Briefly, alongside with the original dynamical system that describes the process, the adjoint system describes the derivative states at each point of the process backward, via the chain rule.
- From it, we can obtain the derivative wrt the initial state, and, in a similar way, wrt the parameters of a function that is modeling dynamics.

Adjoint Sensitivity Method

For now focus on one subinteval $[t_0, t_1]$

Consider optimizing a scalar-valued loss function $L()$, whose input is the result of an ODE solver

$$L(\mathbf{z}(t_1), \mathbf{z}_1^{obs}) = L\left(\mathbf{z}(t_0) + \int_{t_0}^{t_1} f(\mathbf{z}(t), \theta)dt, \mathbf{z}_1^{obs}\right) = L\Big(OSESolve(\mathbf{z}(t_0), f, t_0, t_1, \theta), \mathbf{z}_1^{obs}\Big)$$

Optimizing L requires gradients wrt θ

The first step is to determining how the gradient of the loss depends on the hidden state $\mathbf{z}(t)$ at each instant.

This quantity is called the **adjoint** $\mathbf{a}(t) = \dfrac{\partial L}{\partial \mathbf{z}(t)}$

Its dynamics are given by another ODE, which can be thought of as the instantaneous analog of the chain rule:



$$\frac{d\mathbf{a}(t)}{dt} = -\mathbf{a}(t)^T \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}}$$

(proof later ◆)

We can compute $\dfrac{\partial L}{\partial \mathbf{z}(t_0)}$ by another call to an ODE solver. This solver must run backwards, starting from the initial value of $\dfrac{\partial L}{\partial \mathbf{z}(t_1)}$.

Adjoint Sensitivity Method

One complication is that solving this ODE requires the knowing value of $\mathbf{z}(t)$ along its entire trajectory.

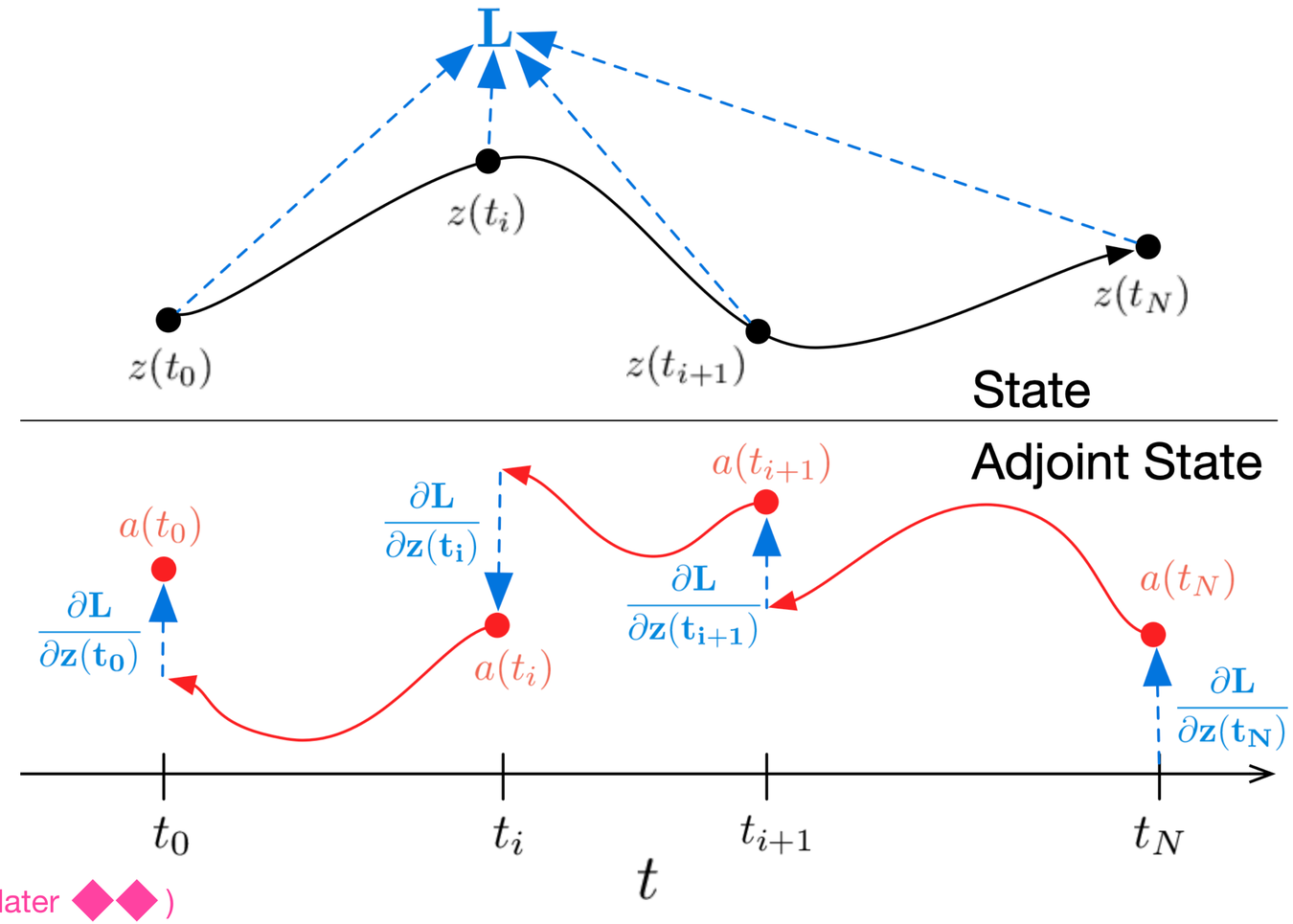
However, we can simply recompute $\mathbf{z}(t)$ backwards in time together with the adjoint, starting from its final value $\mathbf{z}(t_1)$.

Composing the two ODEs

However, computing the gradients with respect to the parameters θ requires evaluating a third integral, which depends on both $\mathbf{z}(t)$ and $\mathbf{a}(t)$:

$$\frac{dL}{d\theta} = -\int_{t_1}^{t_0} \mathbf{a}(t)^T \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \theta} dt$$

(proof later ◆◆)



The vector-Jacobian products $\mathbf{a}(t)^T \frac{\partial f(\mathbf{z})}{\partial \mathbf{z}}$ and $\mathbf{a}(t)^T \frac{\partial f(\mathbf{z})}{\partial \theta}$ can be efficiently evaluated by (reverse) automatic differentiation, at a time cost similar to that of evaluating f. All integrals for solving $\mathbf{z}$, $\mathbf{a}$ and $\frac{dL}{d\theta}$ can be computed in a single call to an ODE solver, which concatenates the original state, the adjoint, and the other partial derivatives into a single vector.

Starting from

$$\mathbf{g}(t_0) := \frac{dL}{d\theta} = -\int_{t_1}^{t_0} \mathbf{a}(t)^T \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \theta} dt$$

Define

$$\mathbf{g}(t) := -\int_{t_1}^{t} \mathbf{a}(\tau) \cdot \frac{\partial f(\mathbf{z}(\tau), \tau, \theta)}{\partial \theta} d\tau$$



$$\frac{d\mathbf{g}(t)}{dt} = -\mathbf{a}(t)^T \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \theta}$$

$\mathbf{g}(t_1) = 0$ integration interval has zero length

When we are only interested in the gradient $dL/d\theta$ for a scalar objective L the **adjoint method** allows us to compute this gradient without computing the full Jacobian $\partial \mathbf{z}/\partial\theta$ (which is a huge matrix).

1

2

3

Augmented System

$$\begin{cases} \dfrac{d\mathbf{z}}{dt} = f(\mathbf{z}(t), t, \theta), \\ \dfrac{d\mathbf{a}}{dt} = -\left(\dfrac{\partial f}{\partial \mathbf{z}}\right)^T \mathbf{a}(t), \\ \dfrac{d\mathbf{g}}{dt} = -\left(\dfrac{\partial f}{\partial \theta}\right)^T \mathbf{a}(t). \end{cases}$$

Initial condition $\mathbf{s}(t_1) = \left[\mathbf{z}(t_1), \mathbf{a}(t_1), \mathbf{0}_p\right]$

Augmented State

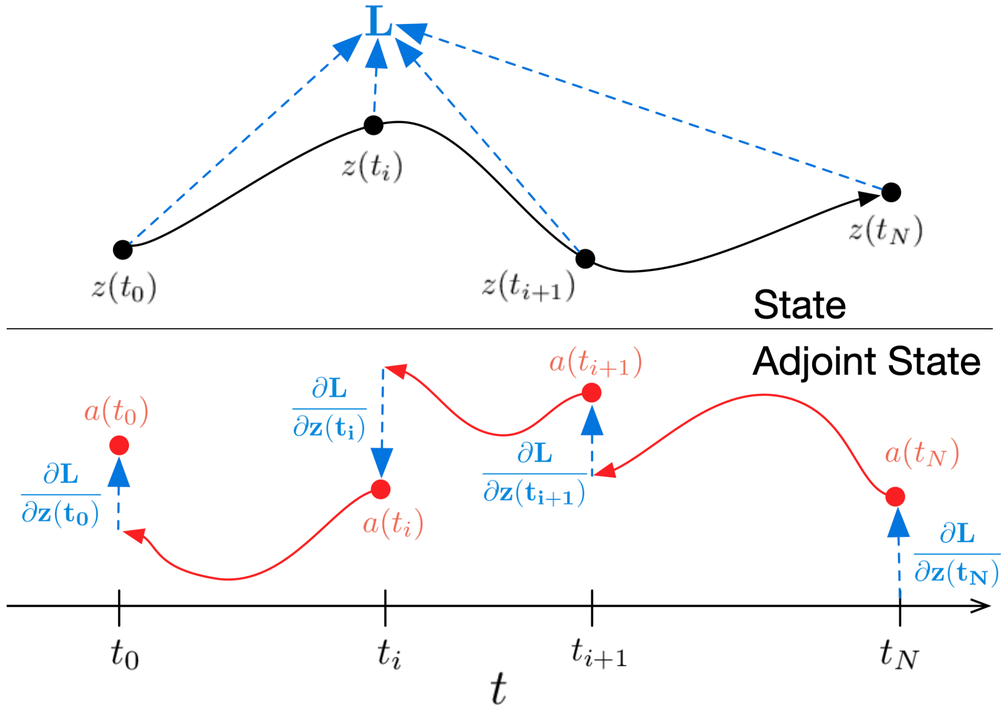
$$\mathbf{s}(t) = \left[\mathbf{z}(t), \mathbf{a}(t), \mathbf{g}(t)\right] = \left[\mathbf{z}(t), \frac{\partial L}{\partial z(t)}, \frac{\partial L}{\partial \theta}\Big|_{[t,\, t_1]}\right]$$

. ODE solver computes all the derivatives at once!

*with a slight abuse of notation, we write $\frac{\partial L}{\partial \theta}\Big|_{[t,t_1]}$ to denote the parameter-gradient contribution accumulated over the interval $[t, t_1]$.

Memo: $\dfrac{\partial L}{\partial \theta}\Big|_{[t_0,t_1]} \equiv \dfrac{\partial L}{\partial \theta}$

Adjoint Sensitivity Method



Pro: we are not explictly computing, nor storing the Jacobian $\frac{\partial \mathbf{z}}{\partial \theta} \in \mathbb{R}^{n \times p}$

Construct the necessary dynamics, and call an ODE solver to compute all gradients at once.

Algorithm 1 Reverse-mode derivative of an ODE initial value problem

Input: dynamics parameters θ, start time t_0, stop time t_1, final state $\mathbf{z}(t_1)$, loss gradient $\partial L / \partial \mathbf{z}(t_1)$

$s_0 = [\mathbf{z}(t_1), \frac{\partial L}{\partial \mathbf{z}(t_1)}, \mathbf{0}_{|\theta|}]$ ▷ Define initial augmented state

def aug_dynamics$([\mathbf{z}(t), \mathbf{a}(t), \cdot], t, \theta)$: ▷ Define dynamics on augmented state

  return $[f(\mathbf{z}(t), t, \theta), -\mathbf{a}(t)^\mathsf{T} \frac{\partial f}{\partial \mathbf{z}}, -\mathbf{a}(t)^\mathsf{T} \frac{\partial f}{\partial \theta}]$ ▷ Compute vector-Jacobian products

$[\mathbf{z}(t_0), \frac{\partial L}{\partial \mathbf{z}(t_0)}, \frac{\partial L}{\partial \theta}] = \text{ODESolve}(s_0, \text{aug_dynamics}, t_1, t_0, \theta)$ ▷ Solve reverse-time ODE

return $\frac{\partial L}{\partial \mathbf{z}(t_0)}, \frac{\partial L}{\partial \theta}$ ▷ Return gradients

Proof of

$$\frac{d\mathbf{a}(t)}{dt} = -\,\mathbf{a}(t)^T \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}}$$

Given $\mathbf{a}(t) = \dfrac{dL}{d\mathbf{z}(t)}$ and $\dfrac{d\mathbf{z}(t)}{dt} = f(\mathbf{z}(t), t, \theta)$

The gradient of a hidden layer $\mathbf{h}_t$ depends on the gradient from the next layer $\mathbf{h}_{t+1}$ by chain rule

$$\frac{\partial \mathscr{L}}{\partial \mathbf{h}_t} = \frac{\partial \mathscr{L}}{\partial \mathbf{h}_{t+1}} \frac{\partial \mathbf{h}_{t+1}}{\partial \mathbf{h}_t}$$

$$\begin{aligned}
\frac{d\mathbf{a}(t)}{dt} &= \lim_{\varepsilon \to 0^+} \frac{\mathbf{a}(t+\varepsilon) - \mathbf{a}(t)}{\varepsilon} \\
&= \lim_{\varepsilon \to 0^+} \frac{\mathbf{a}(t+\varepsilon) - \mathbf{a}(t+\varepsilon)\frac{\partial}{\partial \mathbf{z}(t)} T_\varepsilon(\mathbf{z}(t))}{\varepsilon} \\
&= \lim_{\varepsilon \to 0^+} \frac{\mathbf{a}(t+\varepsilon) - \mathbf{a}(t+\varepsilon)\frac{\partial}{\partial \mathbf{z}(t)} \left(\mathbf{z}(t) + \varepsilon f(\mathbf{z}(t), t, \theta) + \mathcal{O}(\varepsilon^2)\right)}{\varepsilon} \\
&= \lim_{\varepsilon \to 0^+} \frac{\mathbf{a}(t+\varepsilon) - \mathbf{a}(t+\varepsilon)\left(I + \varepsilon \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}(t)} + \mathcal{O}(\varepsilon^2)\right)}{\varepsilon} \\
&= \lim_{\varepsilon \to 0^+} \frac{-\varepsilon \mathbf{a}(t+\varepsilon)\frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}(t)} + \mathcal{O}(\varepsilon^2)}{\varepsilon} \\
&= \lim_{\varepsilon \to 0^+} \left(-\mathbf{a}(t+\varepsilon)\frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}(t)} + \mathcal{O}(\varepsilon)\right) \\
&= -\,\mathbf{a}(t)\frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}(t)}\,.
\end{aligned}$$

The transformation after an ε change in time

$$\mathbf{z}(t+\varepsilon) = \mathbf{z}(t) + \int_t^{t+\varepsilon} f(\mathbf{z}(s), s, \theta)\, ds = T_\varepsilon(\mathbf{z}(t), t)$$

and chain rule can also be applied

$$\frac{dL}{d\mathbf{z}(t)} = \frac{\partial L}{\partial \mathbf{z}(t+\varepsilon)} \frac{\partial \mathbf{z}(t) + \varepsilon}{\partial \mathbf{z}(t)} \quad \text{or} \quad \mathbf{a}(t) = \mathbf{a}(t+\varepsilon)\frac{\partial T_\varepsilon(\mathbf{z}(t), t)}{\partial \mathbf{z}(t)}$$

Proof of

$$\frac{dL}{d\theta} = -\int_{t_1}^{t_0} \mathbf{a}(t)^T \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \theta} dt$$

The key idea is to understand how a small perturbation of the parameters affects the entire trajectory and, consequently, the final loss.

The loss depends on θ indirectly through the trajectory $\mathbf{z}(t)$

- Define the sensitivity matrix $S(t) = \frac{\partial \mathbf{z}(t)}{\partial \theta} \in \mathbb{R}^{n \times p}$.
- Differentiate the original ODE with respect to θ: $\frac{d}{dt}\left(\frac{\partial \mathbf{z}(t)}{\partial \theta}\right) = \frac{\partial}{\partial \theta} f(\mathbf{z}(t), t, \theta)$.
- Applying the chain rule: $\frac{dS(t)}{dt} = \frac{\partial f}{\partial \mathbf{z}} S(t) + \frac{\partial f}{\partial \theta}$ with $S(t_0) = 0$, because the initial condition does not depend on θ.

  Recall the adjoint equation $\frac{d\mathbf{a}(t)}{dt} = -\mathbf{a}(t)^T \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \mathbf{z}}$.

- Differentiate the product $\mathbf{a}(t)^T S(t)$

  Applying the product rule: $\frac{d}{dt}(\mathbf{a}^T S) = \dot{\mathbf{a}}^T S + \mathbf{a}^T \dot{S} = -\mathbf{a}^T \frac{\partial f}{\partial \mathbf{z}} S + \mathbf{a}^T \left(\frac{\partial f}{\partial \mathbf{z}} S + \frac{\partial f}{\partial \theta}\right) = \mathbf{a}^T \frac{\partial f}{\partial \theta}$.

- **Integrate over time:** Assuming $\mathbf{z}(t_0)$ is independent of θ, so $S(t_0) = 0$:

$$\int_{t_0}^{t_1} \mathbf{a}^T \frac{\partial f}{\partial \theta} dt = \left[\mathbf{a}^T S\right]_{t_0}^{t_1} = \mathbf{a}(t_1)^T S(t_1) = \frac{\partial L}{\partial \mathbf{z}(t)} \frac{\partial \mathbf{z}(t)}{\partial \theta} = \frac{dL}{d\theta}.$$

From Discrete Backpropagation to Continuous-Time Gradients

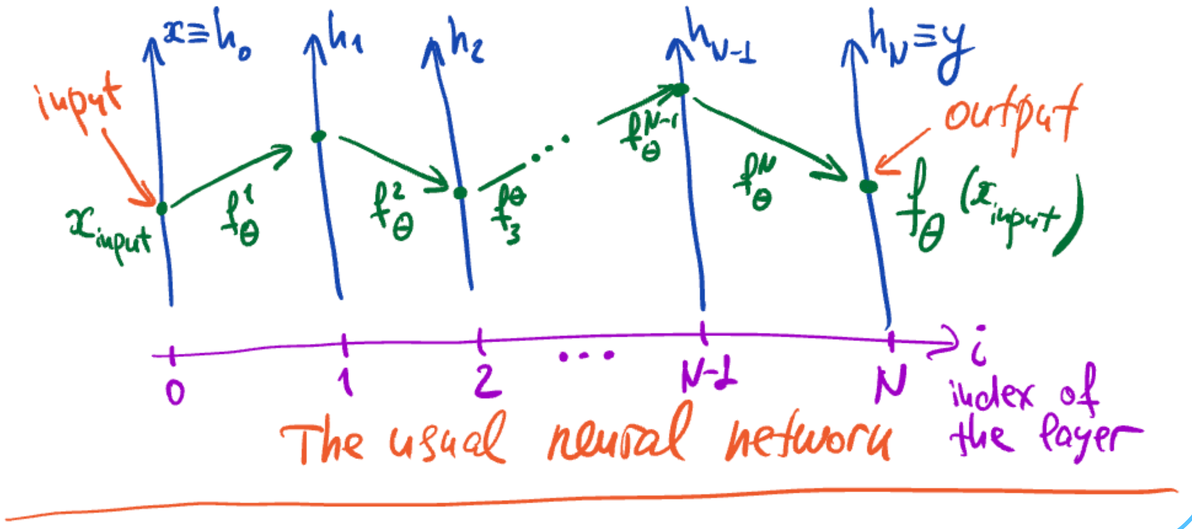
In a standard neural network, the output is obtained through a sequence of transformations:

$$\mathbf{h}_{i+1} = f_\theta^i(\mathbf{h}_i)\,.$$

The gradient of the loss with respect to shared parameters θ is obtained by summing the contributions from all layers:

$$\frac{dL}{d\theta} = \sum_{i=0}^{N-1} \mathbf{a}_{i+1}^T \frac{\partial f_\theta^i(\mathbf{h}_i)}{\partial \theta} \text{ where } \mathbf{a}_{i+1} = \frac{\partial L}{\partial \mathbf{h}_{i+1}} \text{ represents the}$$

gradient propagated backward from the subsequent layers.

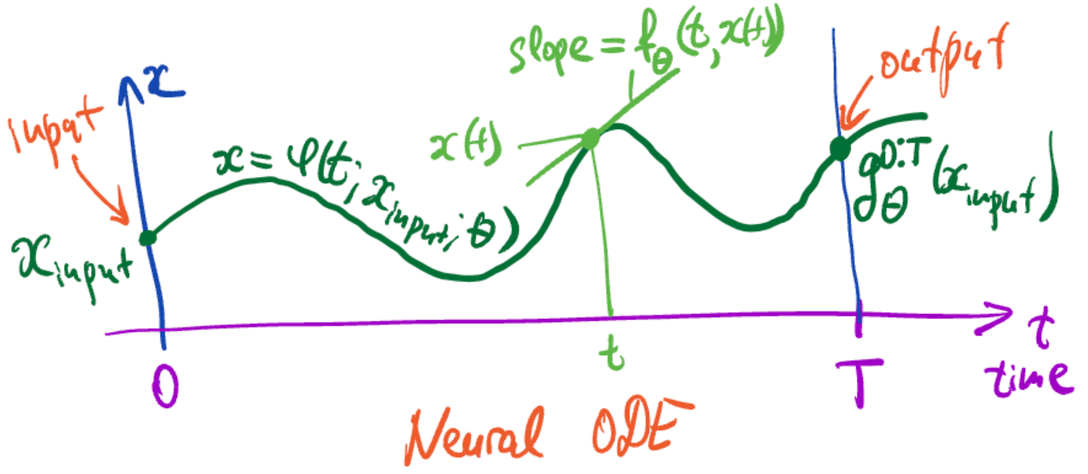


In a Neural ODE, discrete layers are replaced by a continuous trajectory.

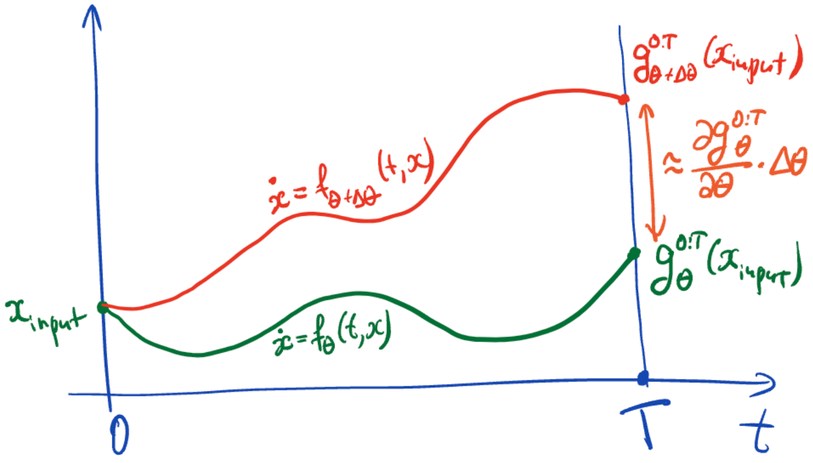
The sum of gradient contributions becomes an integral over time:

$$\frac{dL}{d\theta} = \int_{t_0}^{t_1} \mathbf{a}(t)^T \frac{\partial f(\mathbf{z}(t), t, \theta)}{\partial \theta}\, dt\,.$$

Key idea: Backpropagation through discrete layers becomes the accumulation of infinitesimal gradient contributions along a continuous trajectory.



1. Parameter perturbation: A small change in the parameters, $\theta \to \theta + \Delta\theta$, modifies the vector field and therefore the entire trajectory. The resulting change in the final state is approximately $\Delta\mathbf{z}(t_1) \approx \dfrac{\partial \mathbf{z}(t_1)}{\partial \theta}\Delta\theta$.

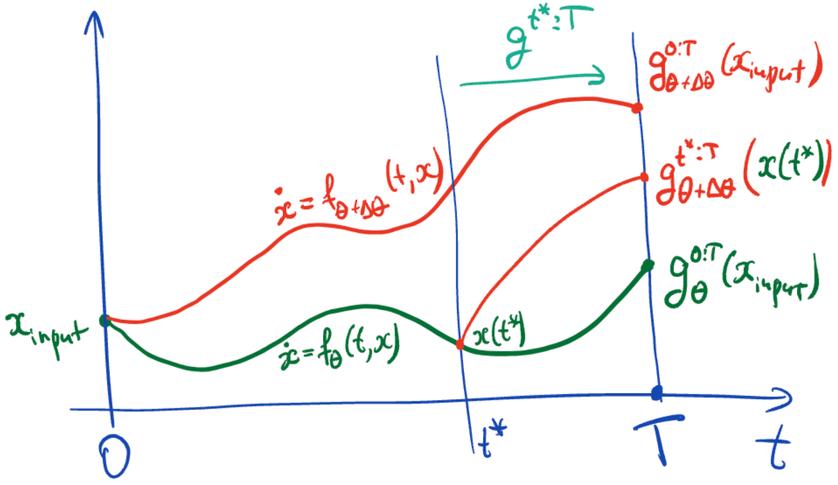


The initial and the perturbed solutions

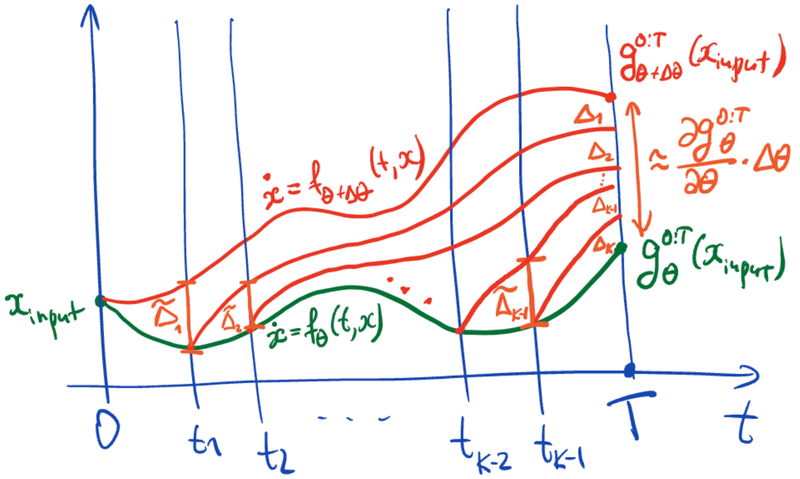
2. State perturbation at an intermediate time

A small perturbation $\Delta\mathbf{z}(t^*)$ at time t^* affects the subsequent trajectory and the final loss: $\Delta L \approx \mathbf{a}(t^*)^T \Delta\mathbf{z}(t^*)$.

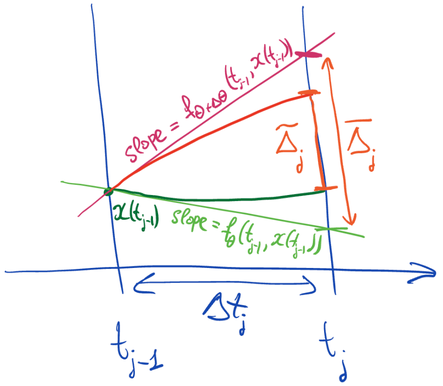
The adjoint $\mathbf{a}(t^*)$ measures the sensitivity of the final loss to changes in the state at time t^*.



Additional perturbed trajectory



Many perturbed trajectories



Approximation trajectories with tangents

3. Accumulating perturbations over time

A parameter perturbation introduces an infinitesimal state change during a small interval dt: $d\mathbf{z}_{\text{pert}} \approx \dfrac{\partial f}{\partial \theta}\Delta\theta\, dt$.

Its contribution to the final loss is $dL_{\text{pert}} \approx \mathbf{a}(t)^T \dfrac{\partial f}{\partial \theta}\Delta\theta\, dt$.

Integrating over the trajectory gives $\dfrac{dL}{d\theta} = \displaystyle\int_{t_0}^{t_1} \mathbf{a}(t)^T \frac{\partial f}{\partial \theta}\, dt$.

Key idea: The adjoint measures how perturbations introduced at each time affect the final loss, allowing us to accumulate their contributions without explicitly computing the full state–parameter sensitivity matrix.

Adjoint Sensitivity with Multiple Observations

Consider a Neural ODE with observations at $t_1 < \cdots < t_N = T$:

$$\dot{\mathbf{z}}(t) = f(\mathbf{z}(t), t, \theta), \qquad L = \sum_{i=1}^{N} L_i(\mathbf{z}(t_i))$$

How do we compute the gradients?

We proceed backward, starting from the last observation:

- At t_3: Initialize the adjoint with the gradient of L_3.
- At t_2: The adjoint already carries the influence of L_3. We add the gradient of L_2.
- At t_1: The adjoint now carries the influence of L_2 and L_3. We add the gradient of L_1.
- At t_0: We obtain the sensitivity of the total loss to the initial state.

Key idea: As we move backward, the adjoint progressively accumulates the influence of all observations.

Consider the state at an intermediate observation time t_2.

A small perturbation of $\mathbf{z}(t_2)$ influences the total loss in two ways:

- **Directly:** It changes the local loss L_2.
- **Indirectly:** It changes the subsequent trajectory and therefore the future loss L_3.

Hence, the total sensitivity is

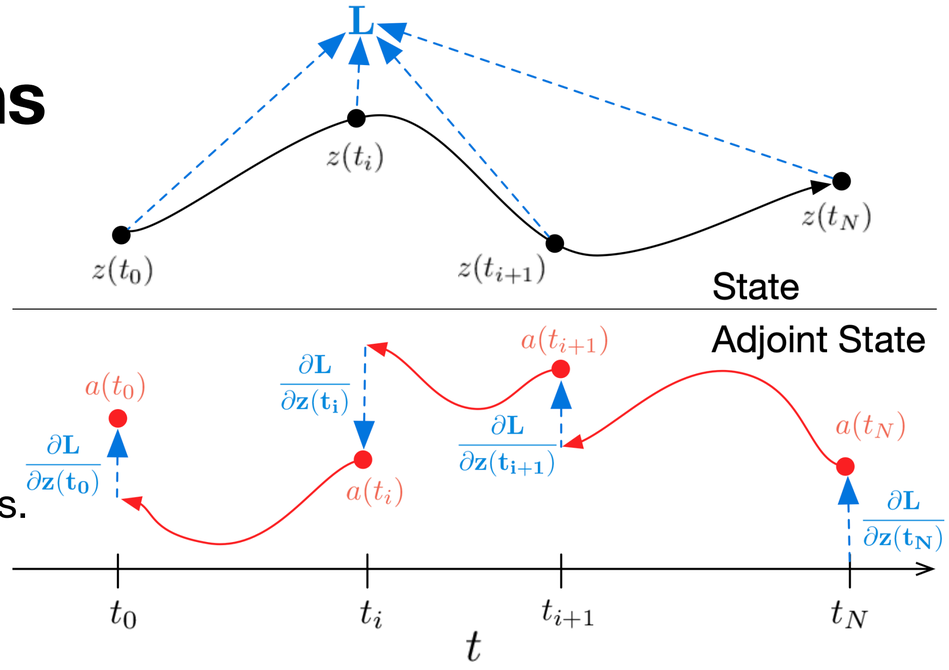
$$\underbrace{\mathbf{a}(t_2^-)}_{\text{Total sensitivity}} = \underbrace{\mathbf{a}(t_2^+)}_{\text{Future losses}} + \underbrace{\nabla_{\mathbf{z}(t_2)} L_2}_{\text{Current loss}}$$

Between observations, the adjoint follows its usual differential equation: $\dfrac{d\mathbf{a}(t)}{dt} = -\left(\dfrac{\partial f}{\partial \mathbf{z}}\right)^T \mathbf{a}(t)\,.$

Analogy with standard back-propagation: Imagine a neural network with losses attached to intermediate layers. When back-propagation reaches one of these layers, we add the local loss gradient to the gradient propagated from subsequent layers.

Takeaway: No new adjoint equation is needed. We simply add the local loss gradient whenever backward integration encounters an observation.

Adjoint Sensitivity with Multiple Observations



General rule

For a loss evaluated at multiple observation times, $L = \sum_{i=1}^{N} L_i(\mathbf{z}(t_i))$

integrate the augmented adjoint system backward between consecutive observations.
At each observation time t_i:

$$\mathbf{a}(t_i^-) = \mathbf{a}(t_i^+) + \nabla_{\mathbf{z}(t_i)} L_i,$$

$$\mathbf{g}(t_i^-) = \mathbf{g}(t_i^+)\,.$$

- Adjoint $\mathbf{a}$: Receives the gradient of the current loss, in addition to the influence of future losses.
- Gradient accumulator $\mathbf{g}$: Is not updated at observation times. It continuously accumulates parameter-gradient contributions during backward integration.

Why is $\mathbf{g}$ unchanged? The local loss depends on θ only through the state $\mathbf{z}(t_i)$. Its effect on the parameter gradient is **already accounted for by the updated adjoint** during subsequent backward integration.

Memo: $\frac{d\mathbf{g}}{dt} = -\left(\frac{\partial f}{\partial \theta}\right)^T \mathbf{a}(t)\,.$

Important: If L_i depends explicitly on θ, we must also add its direct parameter gradient to $\mathbf{g}$ at t_i.

Variational Equations

There is an easy way to compute the the derivates of the solution of an ODE with respect to parameters → **Variational Equations**

$$\begin{cases} \dfrac{d\mathbf{z}}{dt} = f(\mathbf{z}, t, \theta), \\ \mathbf{z}(0) = \mathbf{z}_0 \,. \end{cases}$$

DIRECT SENSITIVITY ANALYSIS

Goal: find sensitivity of $\mathbf{z}(t)$ wrt θ

$$S(t) := \frac{\partial \mathbf{z}(t)}{\partial \theta} \in \mathbb{R}^{n \times p} \,.$$

Differentiate both sides of $\dfrac{d\mathbf{z}}{dt} = f(\mathbf{z}, t, \theta)$ wrt $\dfrac{\partial}{\partial \theta}$

$$\frac{\partial}{\partial \theta}\left(\frac{d\mathbf{z}}{dt}\right) = \frac{\partial f}{\partial \mathbf{z}}\frac{\partial \mathbf{z}}{\partial \theta} + \frac{\partial f}{\partial \theta}$$

$$=$$

$$\frac{d}{dt}\left(\frac{\partial \mathbf{z}}{\partial \theta}\right)$$

$$\longrightarrow$$

$$\begin{cases} \dfrac{dS}{dt} = \dfrac{\partial f}{\partial \mathbf{z}} S + \dfrac{\partial f}{\partial \theta}, \\ S(0) = \dfrac{\partial \mathbf{z}_0}{\partial \theta} \,. \end{cases}$$

Variational Equations

There is an easy way to compute the the derivates of the solution of an ODE with respect to parameters → **Variational Equations**

$$\begin{cases} \dfrac{d\mathbf{z}}{dt} = f(\mathbf{z}, t, \theta), \\ \mathbf{z}(0) = \mathbf{z}_0 \,. \end{cases}$$

DIRECT SENSITIVITY ANALYSIS

Goal: find sensitivity of $\mathbf{z}(t)$ wrt θ

$$S(t) := \frac{\partial \mathbf{z}(t)}{\partial \theta} \in \mathbb{R}^{n \times p} \,.$$

Solve the combined system

$$\begin{cases} \dfrac{d\mathbf{z}}{dt} = f(\mathbf{z}, t, \theta), \\ \dfrac{dS}{dt} = \dfrac{\partial f}{\partial \mathbf{z}} S + \dfrac{\partial f}{\partial \theta}, \\ [\mathbf{z}(0), S(0)] = [\mathbf{z}_0, \dfrac{\partial \mathbf{z}_0}{\partial \theta}] \end{cases}$$

n equations

np additional equations

Solution: $[\mathbf{z}(t), \dfrac{\partial \mathbf{z}}{\partial \theta}]$

Both the ODE solution and its gradient wrt θ.

No need to store intermediate activations!

Computational cost: Direct sensitivity analysis propagates the sensitivity of every state variable with respect to every parameter. For n state variables and $p$parameters, this requires solving n state equations plus np sensitivity equations, for a total of $n(p+1)$scalar ODEs.

Variational Equations

There is an easy way to compute the the derivates of the solution of an ODE with respect to parameters → **Variational Equations**

$$\begin{cases} \dfrac{d\mathbf{z}}{dt} = f(\mathbf{z}, t, \theta), \\ \mathbf{z}(0) = \mathbf{z}_0 \,. \end{cases}$$

DIRECT SENSITIVITY ANALYSIS

Goal: find sensitivity of $\mathbf{z}(t)$ wrt θ

$$S(t) := \frac{\partial \mathbf{z}(t)}{\partial \theta} \in \mathbb{R}^{n \times p} \,.$$

Solve the combined system

$$\begin{cases} \dfrac{d\mathbf{z}}{dt} = f(\mathbf{z}, t, \theta), \\ \dfrac{dS}{dt} = \dfrac{\partial f}{\partial \mathbf{z}} S + \dfrac{\partial f}{\partial \theta}, \\ [\mathbf{z}(0), S(0)] = [\mathbf{z}_0, \dfrac{\partial \mathbf{z}_0}{\partial \theta}] \end{cases}$$

n equations

np additional equations

Computational advantage: The adjoint method replaces the np state sensitivities required by forward sensitivity analysis with an n-dimensional adjoint and a p-dimensional gradient accumulator, resulting in an augmented system of dimension $2n + p$. This is particularly advantageous when $p \gg n$.

Computational cost: Direct sensitivity analysis propagates the sensitivity of every state variable with respect to every parameter. For n state variables and $p$parameters, this requires solving n state equations plus np sensitivity equations, for a total of $n(p + 1)$scalar ODEs.

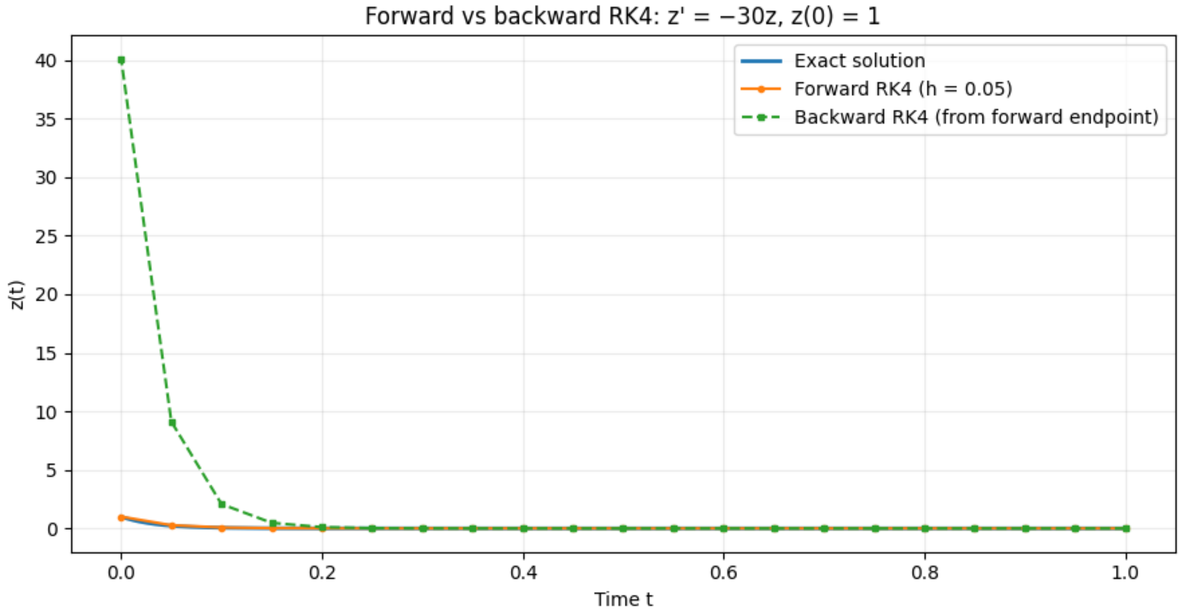
Errors in backward solvers

Forward perturbations decay, but backward perturbations are amplified

Consider an **highly dissipative ODE** (resulting in a stiff trajectory) $\frac{dz}{dt} = -30z$

The exact solution is $z(t) = e^{-30t}$

Let's use RK4 with $\Delta t = 0.05$ first forward from $t = 0$ to $t = 1$ and then backward starting from the final value obtained numerically



RK4 is stable in forward step (for the selected Δt), but the backward reconstruction is highly inaccurate!

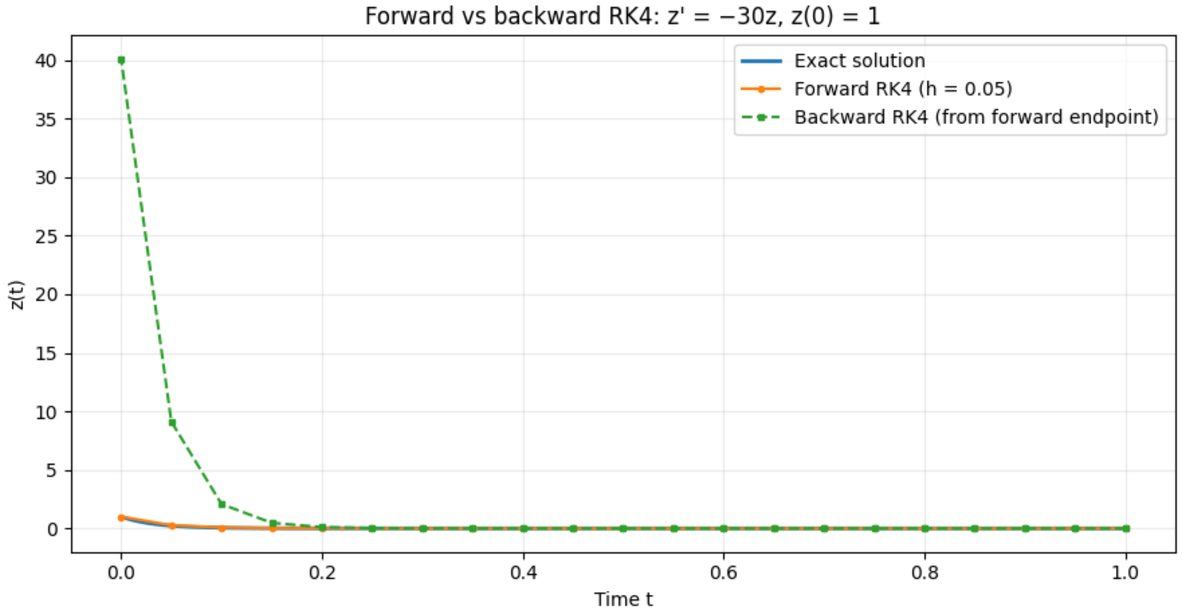
Errors in backward solvers

Forward perturbations decay, but backward perturbations are amplified

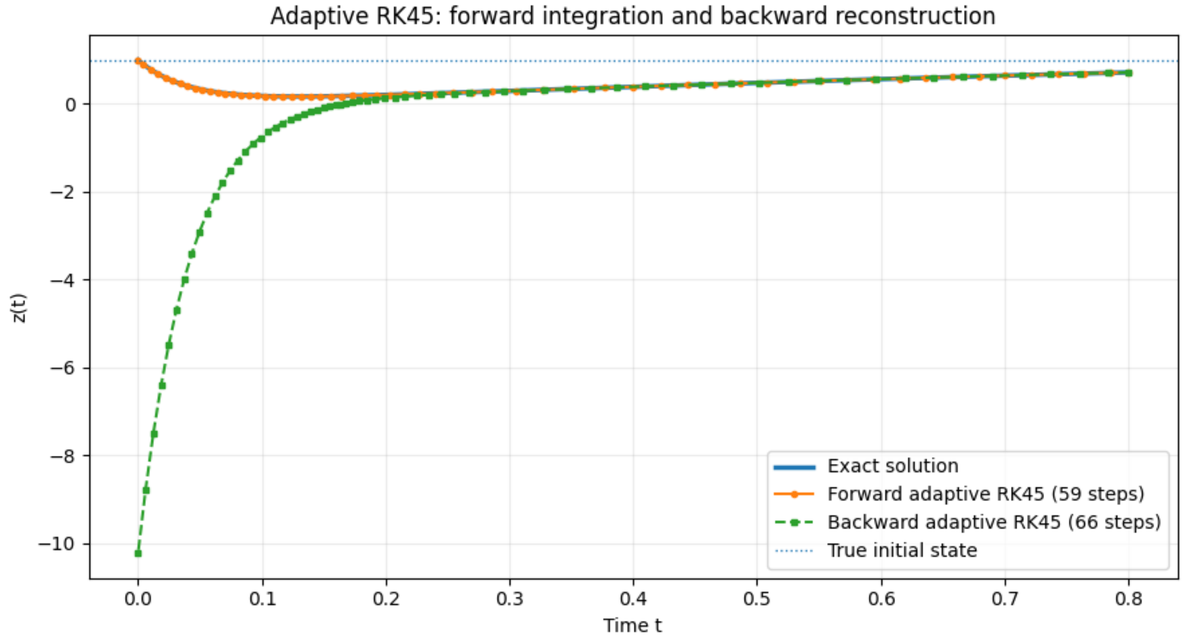
Consider an **highly dissipative ODE** (resulting in a stiff trajectory) $\frac{dz}{dt} = -30z$

The exact solution is $z(t) = e^{-30t}$

Let's use RK4 with $\Delta t = 0.05$ first forward from $t = 0$ to $t = 1$ and then backward starting from the final value obtained numerically



RK4 is stable in forward step (for the selected Δt), but the backward reconstruction is highly inaccurate!



A more adaptive solver, RK45, that automatically adapt the step to control the numerical error is not doing any better.

Backward solvers: checkpoints

example

Checkpointing reduces numerical errors during backward trajectory reconstruction by storing selected states from the forward integration.

$$\frac{dz}{dt} = -30z$$

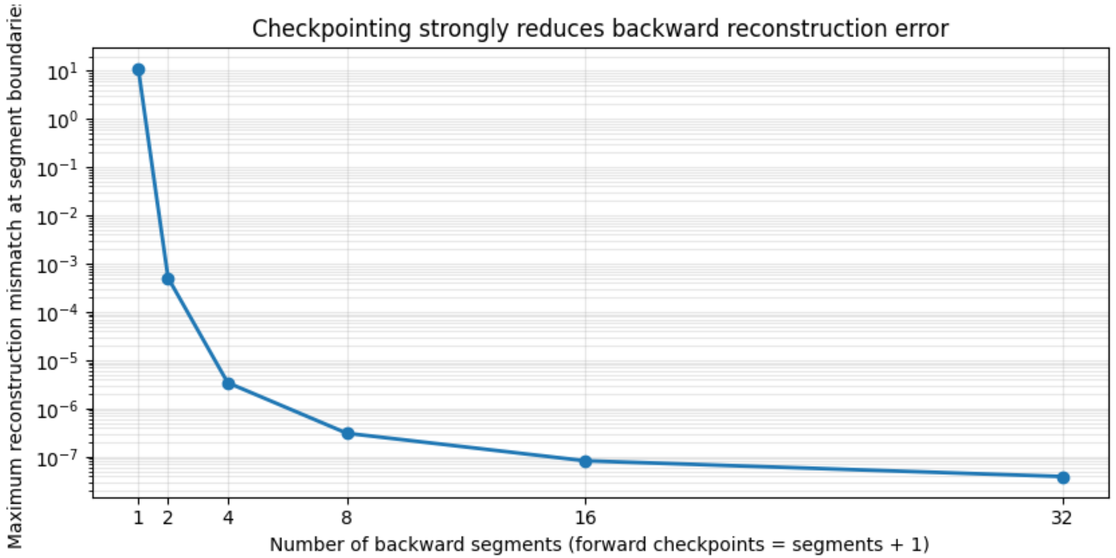
Instead of reconstructing the entire trajectory backward from the final state, the solver integrates over shorter intervals, restarting from the saved checkpoints. This **improves numerical stability at the cost of additional memory.**

```
import torch
from torchdiffeq import odeint_adjoint

# Specify the output times (used as checkpoints)
t = torch.linspace(0, 1, 21)

z = odeint_adjoint(
    model, z0, t,
    method="dopri5",
    rtol=1e-7,
    atol=1e-9
)

loss = loss_fn(z[-1], target)
loss.backward()
```



Checkpointing strongly reduces backward reconstruction error!

In torchdiffeq, odeint_adjoint stores the states at the requested times and uses them to restart backward integration over consecutive intervals.

Note: These checkpoints are not the solver's internal adaptive time steps. More checkpoints can improve backward reconstruction accuracy, but increase memory usage and do not guarantee more accurate gradients.

Error Control

ODE solvers can approximately ensure that the output is within a given tolerance of the true solution. Changing this tolerance changes the behavior of the network. The time spent by the forward call is proportional to the number of function evaluations (Figure 3b), so tuning the tolerance gives us a trade-off between accuracy and computational cost. One could train with high accuracy, but switch to a lower accuracy at test time.

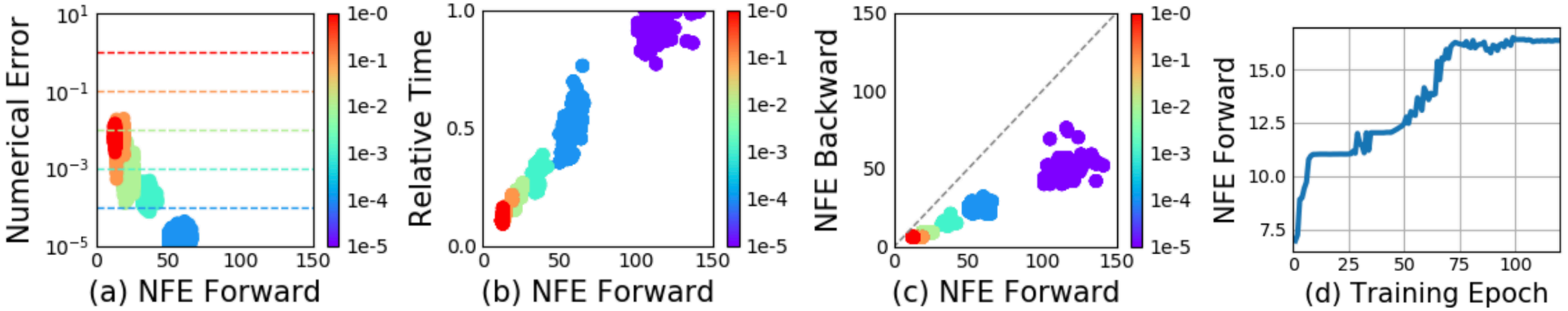


Figure 3: Statistics of a trained ODE-Net. (NFE = number of function evaluations.)

The number of evaluations in the backward pass is roughly half of the forward pass. This suggests that the adjoint sensitivity method is not only more memory efficient, but also more computationally efficient than directly backpropagating through the integrator, because the latter approach will need to backprop through each function evaluation in the forward pass.

Advantages of Neural ODEs

Why using Neural ODEs rather than normal ResNets:

- Memory efficiency: we don’t need to store all the parameters and gradients while backpropagating
- Adaptive computation: we can balance speed and accuracy with the discretization scheme, moreover, having it different while training and inference
- Parameters efficiency: the parameters of nearby “layers” are automatically tied together
- Continuous time series models: continuously-defined dynamics can naturally incorporate data which arrives at arbitrary times

Limitations

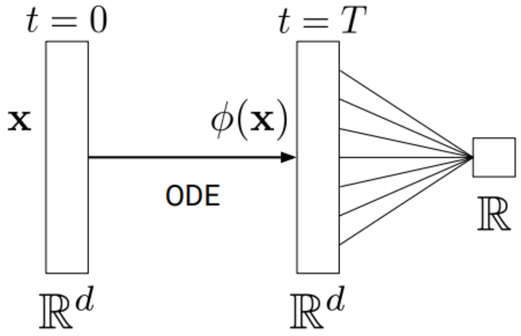
Minibatching: The use of mini-batches is less straightforward than for standard neural networks. One can still batch together evaluations through the ODE solver by concatenating the states of each batch element together, creating a combined ODE with dimension D × K. In some cases, controlling error on all batch elements together might require evaluating the combined system K times more often than if each system was solved individually.

Uniqueness we have an unique solution if the differential equation is uniformly Lipschitz continuous in $\mathbf{z}$ and continuous in t. This holds for our Neural ODE if the neural network has finite weights and uses Lipshitz nonlinearities, such as **tanh** or **relu**.

Setting tolerances. May be problem dependent.

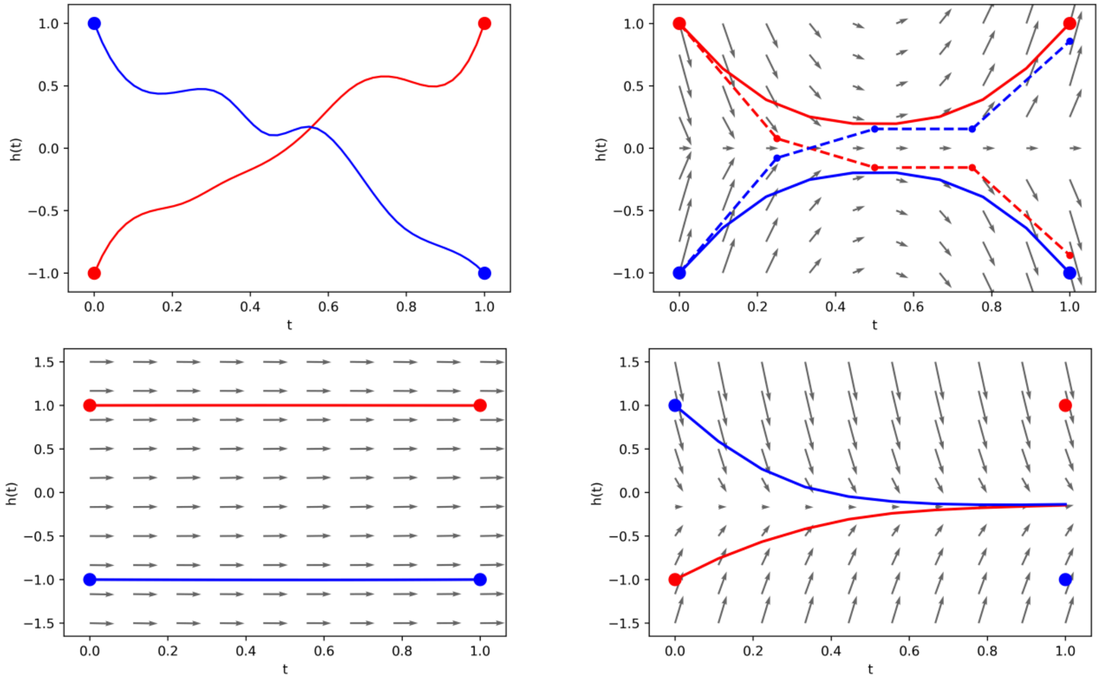
Augmented Neural ODE [paper]

There are simple classes of functions NODEs cannot represent. While it is often possible for NODEs to approximate these functions in practice, the resulting flows are complex and lead to ODE problems that are computationally expensive to solve.



Example 1: Simple function that ODE flows cannot represen: $g_{1d} : \mathbb{R} \to \mathbb{R}$ be a function such that $g_{1d}(-1) = 1$ and $g_{1d}(1) = -1$.

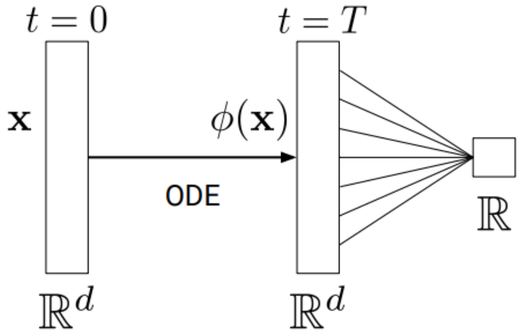
Intuition: the trajectories mapping –1 to 1 and 1 to –1 must intersect each other. However, ODE trajectories cannot cross each other, so the flow of an ODE cannot represent $g_{1d}(x)$.



Remark: Numerical approximations of ODE solutions may exhibit trajectory intersections, even when the exact solutions cannot intersect due to uniqueness. However, accurate numerical solvers should keep these approximations close to the true trajectories. Therefore, numerical intersections are artifacts of discretization, not a way to overcome the topological limitations of Neural ODEs in representing transformations that require intersecting trajectories.

Augmented Neural ODE [paper]

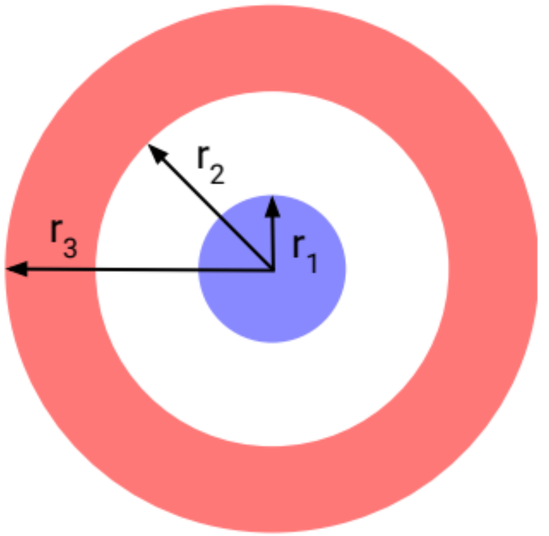
There are simple classes of functions NODEs cannot represent. While it is often possible for NODEs to approximate these functions in practice, the resulting flows are complex and lead to ODE problems that are computationally expensive to solve.



Example 2: $g : \mathbb{R}^d \to \mathbb{R}$

$$\begin{cases} g(\mathbf{x}) = -1 & \text{if } \|\mathbf{x}\| \leq r_1 \\ g(\mathbf{x}) = 1 & \text{if } r_2 \leq \|\mathbf{x}\| \leq r_3, \end{cases}$$

Intuition: In order for the linear layer to map the blue and red points to –1 and 1 respectively, the features $\phi(x)$ for the blue and red points must be linearly separable. Since the blue region is enclosed by the red region, points in the blue region must cross over the red region to become linearly separable, requiring the trajectories to intersect, which is not possible.



$0 < r_1 < r_2 < r_3$

More in general: The feature mapping $\phi(x)$ is a *homeomorphism*, so the features of Neural ODEs *preserve the topology* of the input space.

Augmented Neural ODE [paper]

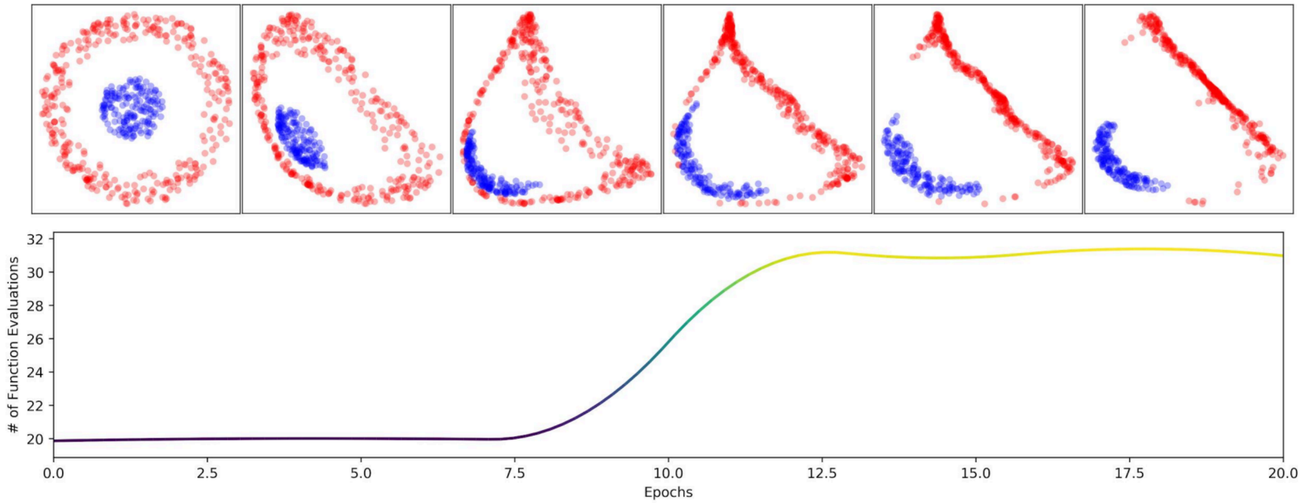
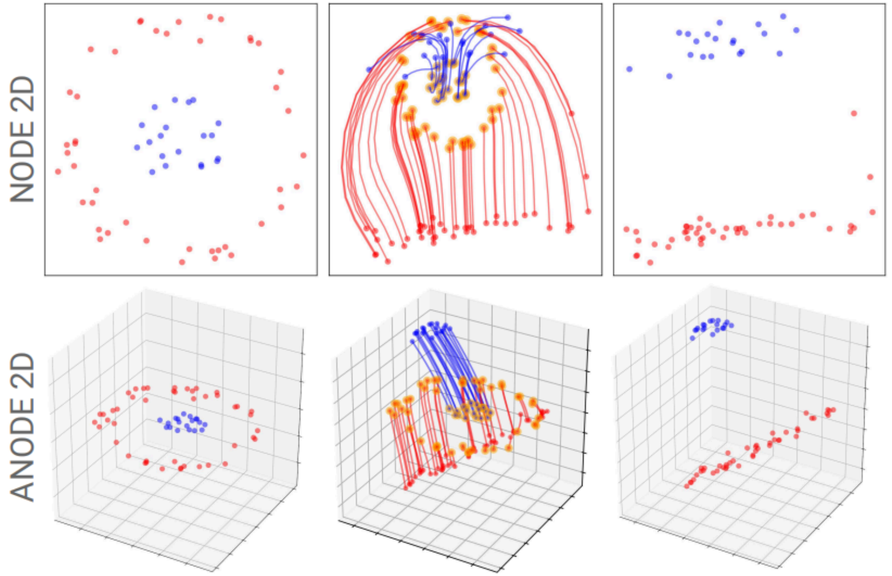


Figure 6: Evolution of the feature space as training progresses and the corresponding number of function evaluations required to solve the ODE. As the ODE needs to break apart the annulus, the number of function evaluations increases.



Augmented Neural ODE [paper]

Augmented Neural ODEs (ANODEs) is a simple extension of NODEs designed to overcome these limitations. ANODEs augment the space on which the ODE is solved, allowing the model to use the additional dimensions to learn more complex functions using simpler flows.

Augment the space on which we learn and solve the ODE from $\mathbb{R}^d$ to $\mathbb{R}^{d+p}$, allowing the ODE flow to lift points into the additional dimensions to avoid trajectories intersecting each other.

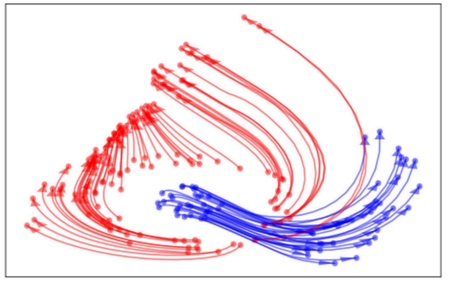
Let $\mathbf{a}(t) \in \mathbb{R}^p$denote a point in the augmented part of the space, we can formulate the augmented ODE problem as

$$\frac{\mathrm{d}}{\mathrm{d}t}\begin{bmatrix}\mathbf{h}(t)\\ \mathbf{a}(t)\end{bmatrix} = \mathbf{f}(\begin{bmatrix}\mathbf{h}(t)\\ \mathbf{a}(t)\end{bmatrix}, t), \qquad \begin{bmatrix}\mathbf{h}(0)\\ \mathbf{a}(0)\end{bmatrix} = \begin{bmatrix}\mathbf{x}\\ \mathbf{0}\end{bmatrix}$$

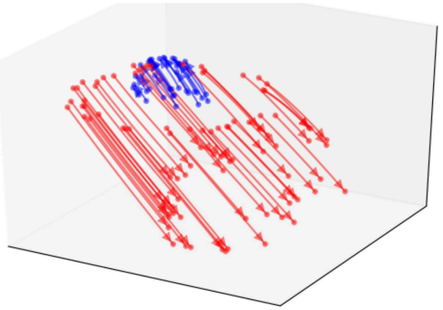
i.e. we concatenate every data point x with a vector of zeros and solve the ODE on this augmented space.

ANODE: more expressive models +

lower computational cost of both forward and backward passes

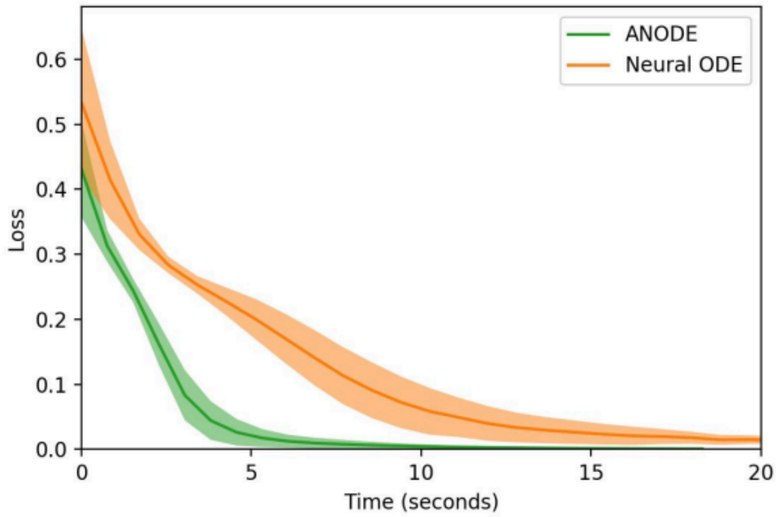


Neural ODE

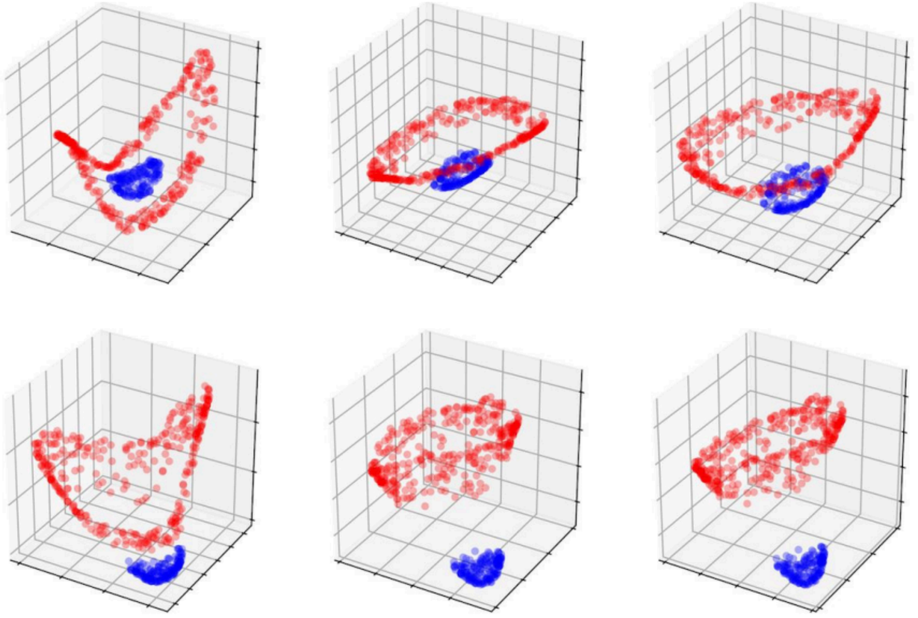


Augmented Neural ODE

Accuracy



Computational cost and number of function evaluations



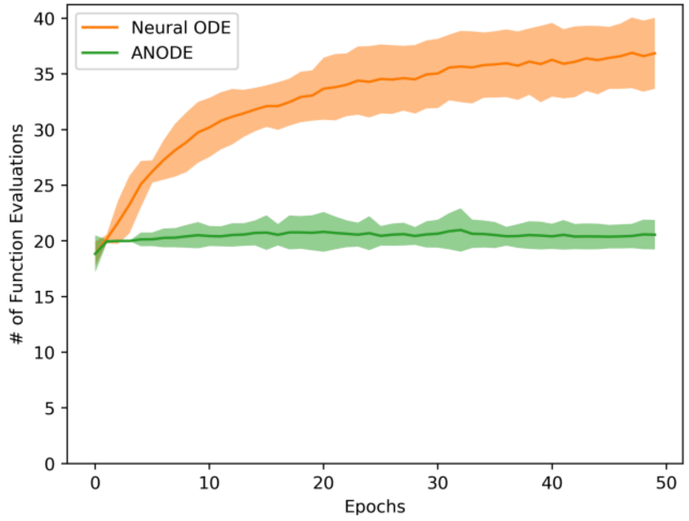
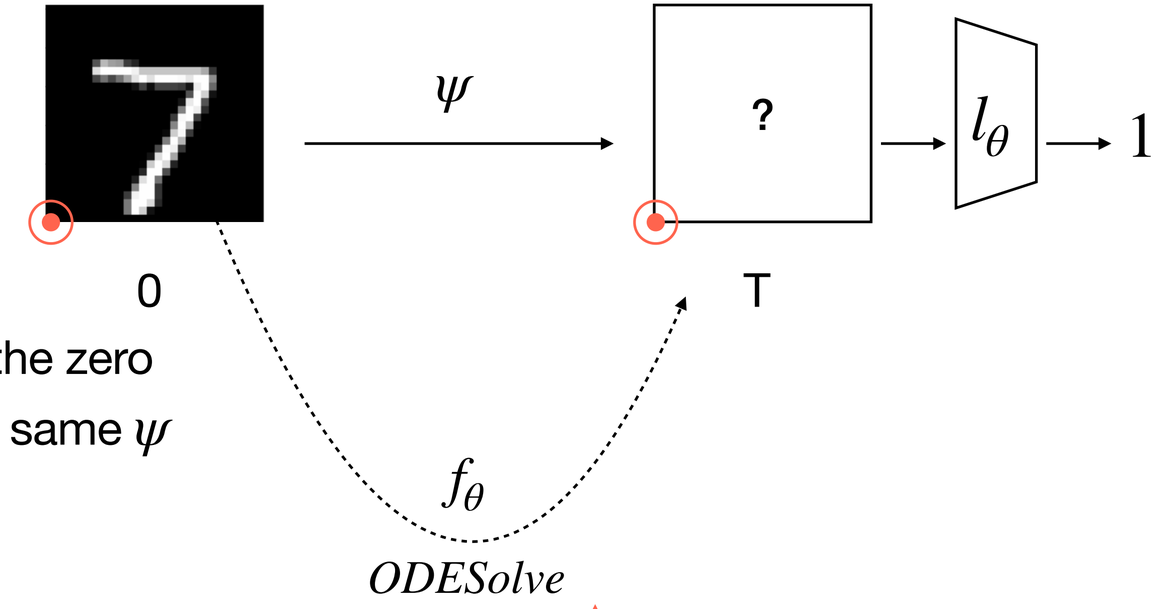


Figure 8: (Left) Evolution of features during training for ANODEs. The top left tile shows the feature space for a randomly initialized ANODE and the bottom right tile shows the features after training. (Right) Evolution of the NFEs during training for NODEs and ANODEs trained on $g(\mathbf{x})$ in $d = 1$.

Augmented Neural ODE

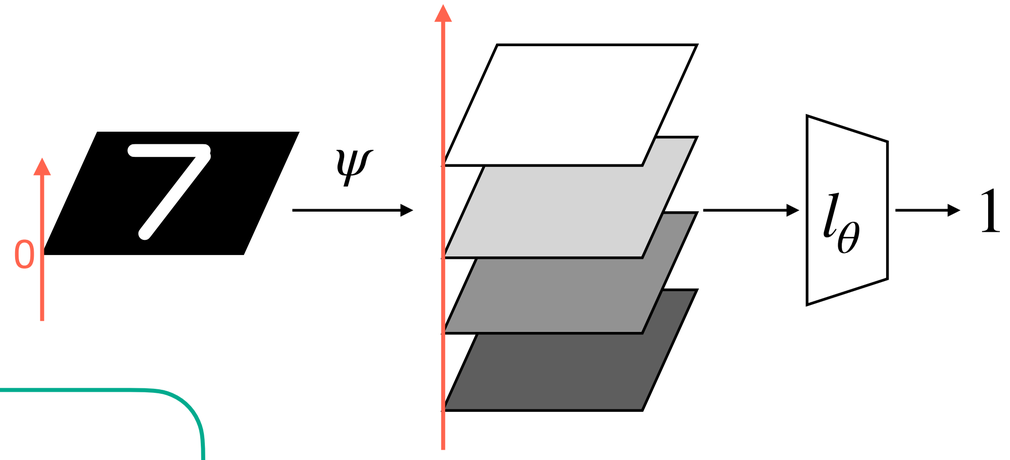


Augmented Model

Returning to the image classification example, taking g_θ as the zero augmentation function (i.e. $g_\theta(y) = (y,0)$) , and keeping the same ψ function, the resulting model becomes:

$$\Psi : \mathbb{R}^{28\times28} \to \mathbb{R}^{28\times28\times1} \to \mathbb{R}^{28\times28\times1} \to \mathbb{R}^{10}$$

$$y \mapsto (y,0) \mapsto \psi((y,0)) \mapsto \textbf{softmax}\Big(l_\theta\big(\psi\big((y,0)\big)\big)\Big)$$



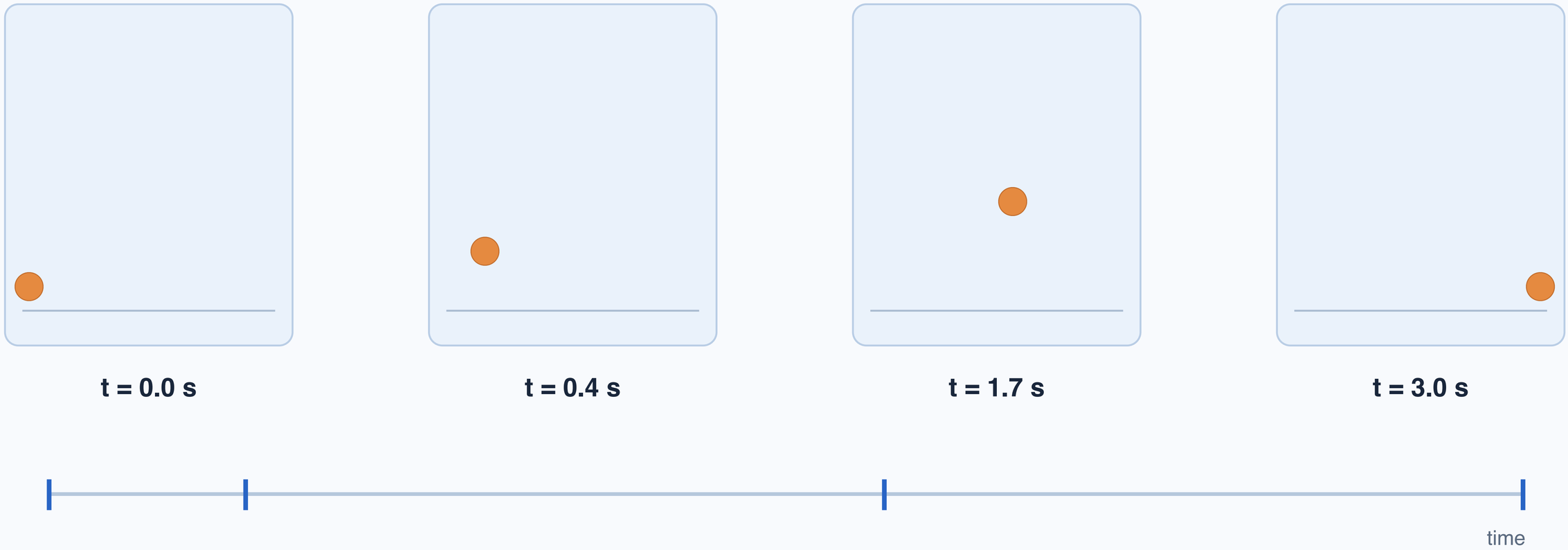
To summarize, in general, a NODE is a model of the form

$$\Psi : \mathbb{R}^{d_{in}} \to \mathbb{R}^{d_l} \to \mathbb{R}^{d_l} \to \mathbb{R}^{d_{out}} \to \mathbb{R}^{d_{out}}$$

$$y \mapsto g_\theta(y) \mapsto \psi(g_\theta(y)) \mapsto l_\theta\big(\psi\big(g_\theta(y)\big)\big) \mapsto \textbf{act}\Big(l_\theta\big(\psi\big(g_\theta(y)\big)\big)\Big)$$

Learning from irregular observation

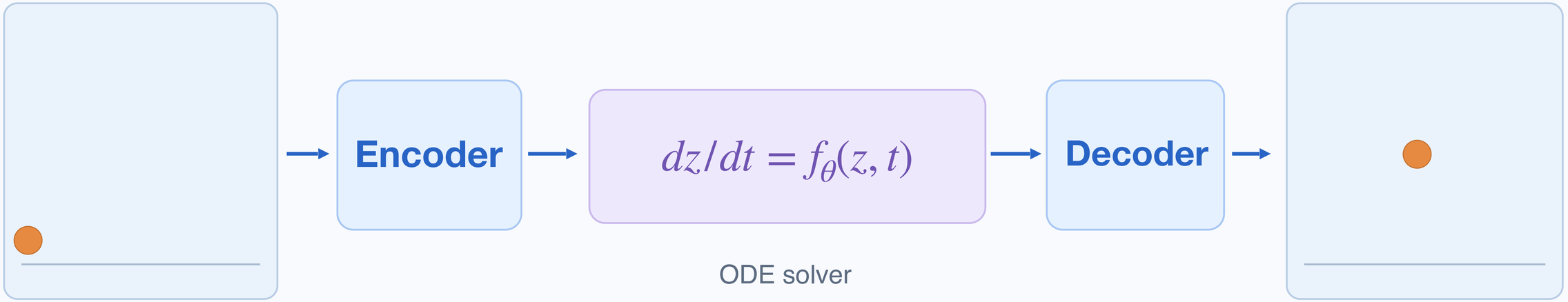
We observe a dynamic scene at only a few non-uniform time points.



GOAL Reconstruct the scene at any intermediate time — including times never observed.

Latent Neural ODE: continuous-time dynamics

Encode an observed frame, evolve its latent state, and decode at any requested time.



Observed image $I(t_0)$

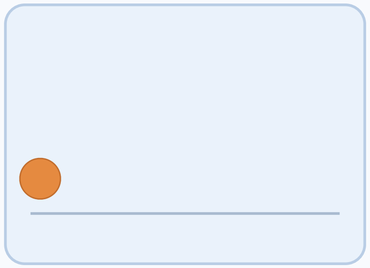
Predicted image $\hat{I}(t)$

Training: compare predicted images $\hat{I}(t_i)$ with all observed snapshots $I(t_i)$.

Interpolate between observations

Once trained, the solver evaluates the trajectory on a denser time grid.

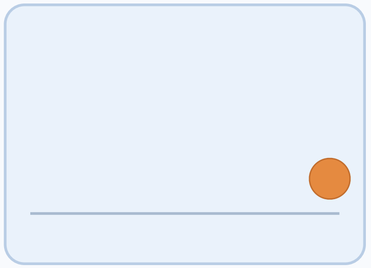
OBSERVED



t = 0.0 s

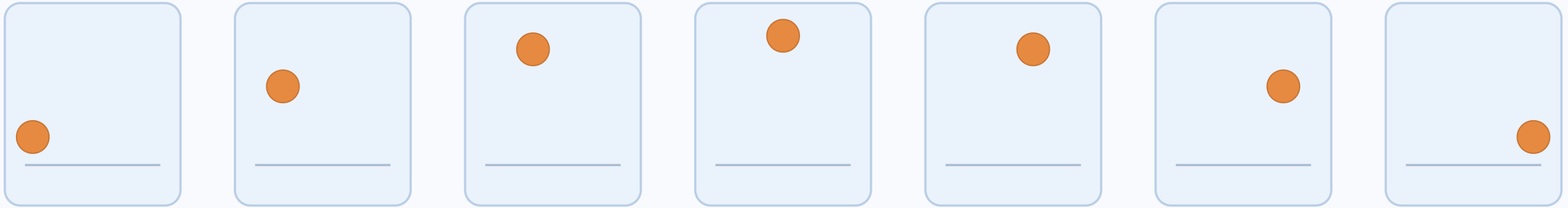


t = 1.7 s



t = 3.0 s

INTERPOLATED



0.0 s 0.5 s 1.0 s 1.5 s 2.0 s 2.5 s 3.0 s

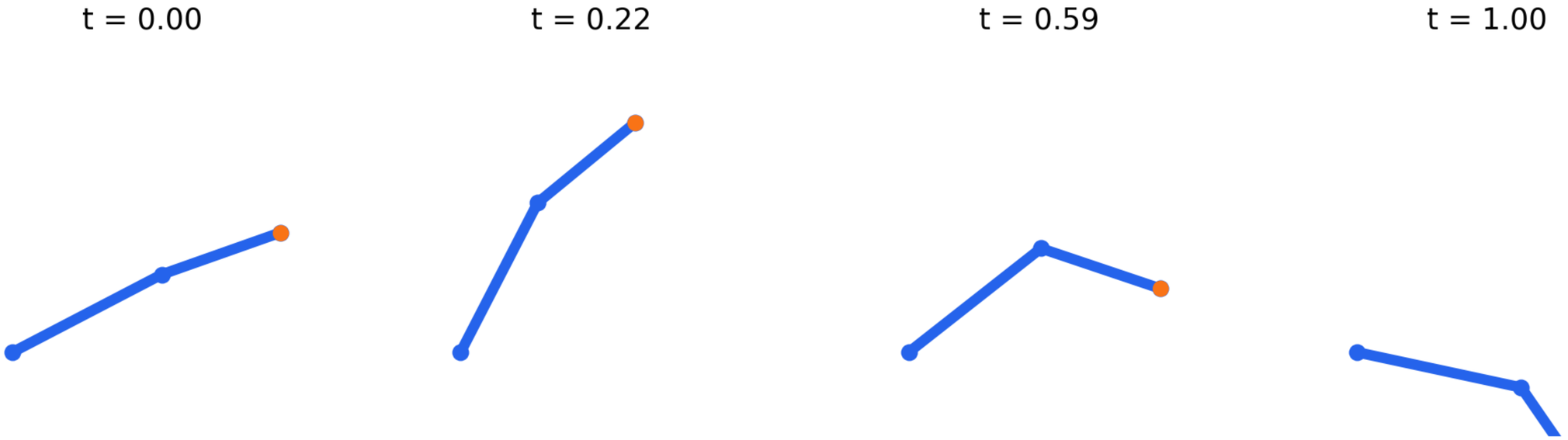
Key idea: irregular sampling during training, continuous-time prediction after training.

Neural ODE: robot-arm snapshots

Learn continuous motion from observations at irregular times

Observed: (t_i, q_i), with unequal time gaps

Question: what is the arm pose at a time we never observed?



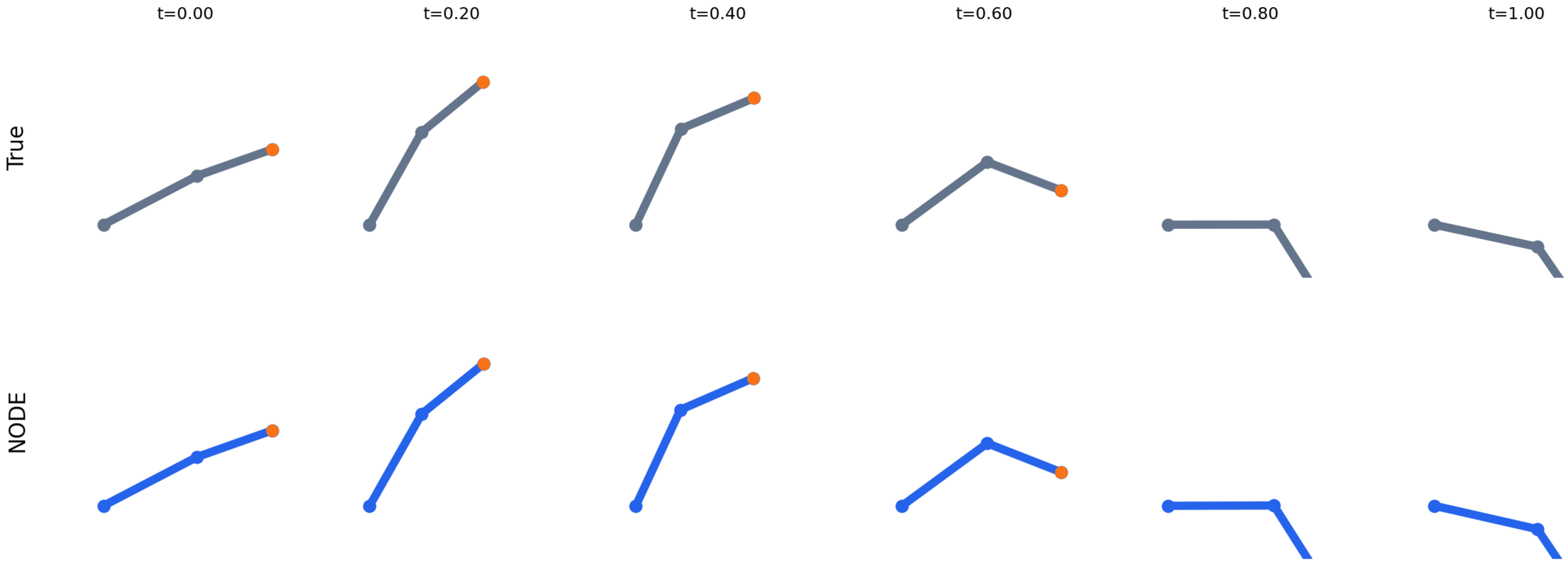
State: $q(t) = [\text{shoulder angle, elbow angle}]$.

Neural ODE: $\frac{dq}{dt} = f_\theta(q, t)$ (integrate from the first observed pose to each timestamp t_i)

Train by minimizing $\Sigma_i \|\hat{q}(t_i) - q_i\|^2$ (the solver handles irregular intervals; gradients flow through integration)

Interpolate unobserved robot poses

Evaluate the learned Neural ODE on a denser time grid



This demo learns from joint angles, not raw pixels. For images: encoder → latent ODE → decoder.

Hands-on

Official Neural ODE code **torchdiffeq**

https://github.com/rtqichen/torchdiffeq/tree/master

PyTorch Implementation of Differentiable ODE Solvers

- First Exercises
- Robot arm example
- Sparse irregular trajectories
- Latent Neural ODE

Coming next…

Physics-Informed ML

- Physics-Informed Neural Networks (PINNs)